

1.1 冯·诺依曼型计算机有哪几个主要部分？其主要设计思想是什么？

存储器、运算器、控制器、输入设备和输出设备，

1、计算机采用二进制逻辑 2、程序存储执行

1.5、1.6进制的转换，主要考大家计算。（十进制、二进制、八进制、十六进制之间的转换。）



1.8 已知：

1) $[x_1]_{\#} = 0000\ 0001$, $[y_1]_{\#} = 1111\ 1111$

2) $[x_2]_{\#} = 0101\ 1110$, $[y_2]_{\text{补}} = 0011\ 0111$

3) $[x_3]_{\text{补}} = 1001\ 1110$, $[y_3]_{\text{补}} = 1100\ 0101$

4) $[x_4]_{\text{补}} = 0110\ 1110$, $[y_4]_{\#} = 1000\ 0100$

请计算 $[x_i]_{\#} + [y_i]_{\#}$, ($i = 1 \sim 4$), 并判断结果是否出现溢出？

00000000b 未溢出

10010101b 溢出

01100011b 溢出

11110010b 未溢出

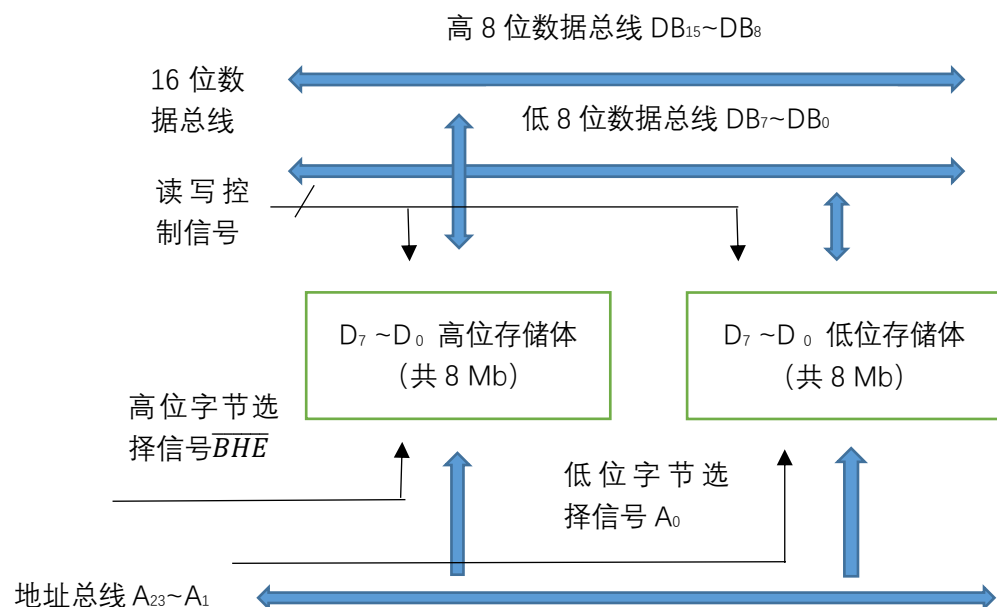
→ 前两位一样与否。

第 2 章作业

2.6 某 16 位计算机系统的数据总线位宽为 16 位，但是地址总线为 24 位，内存按照字节组织，该计算机系统的内存地址空间是多少？如果希望一次能够传送一个字（16 位），或者只传送这个字中的高 8 位或低 8 位，存储器应该如何组织？请画出存储器与总线连接的草图。

24 位地址总线对应的内存地址空间为 $2^{24}=16\text{MB}$

一般来说，内存按字节组织，因此 16MB 的存储系统分成两个 8MB 的存储空间，分别存储高 8 位和低 8 位，使得能够传送一个字（16 位），或者只传送这个字中的高 8 位或低 8 位，草图如下：



2.10 什么是微指令？什么是微程序？控制 ROM 的作用是什么？

微指令：微操作码+执行顺序控制位。对每个微操作进行编码形成微操作码，使得多个微操作序贯执行完成。

微程序：若干条微指令有序排列构成微程序。

控制 ROM：存放所有指令对应的微程序。指令执行时对应的微指令会从控制 ROM 中逐条读出。

2.12 指令采用固定长度编码有哪些优点？也会带来哪些问题？

讲义 2.4.1

优点：

定长指令每条指令长度是固定的，在数据总线上一次可将一条完整的指令送往指令寄存器。

问题：

指令字的一部分要用于表示操作码，其余的地址码部分还需要指明两个寄存器操作数 R_d , R_s ，指令字中可用于表示立即数的编码位数所剩无几。因此只能对指令中的立即数实行某种限制。

32 位定长指令的地址码无法直接给出 32 位内存地址，只能采用间接方式表示。

2.13 请参照例 2.1，分步骤写出第 2 条数据存储指令“LDR $R_1, [R_3]$ ”的执行过程。

第二条指令 PC 值要加 4

1. 程序计数器的内容 $0x20000004$ 送到地址形成部件，地址形成部件产生的地址信号经地址缓冲器/驱动器和地址总线，被送到地址译码器进行译码，得到寻址指令存放的内存单元。
2. 操作控制器发出读控制信号，将 $0x20000004$ 单元的内容“LDR $R_1, [R_3]$ ”读出，该指令是取指操作，于是该指令经过数据总线被存入指令寄存器 IR。
3. PC 自动加 4，指向下一条指令的存放地址。
4. 指令译码器 ID 对指令操作码进行译码，操作控制器 OC 按照操作时序发出相应的控制信号。
5. 指令的地址码部分表明，存放源操作数的指令地址位于 R_3 寄存器中，目的操作数是 R_1 寄存器。
6. 在操作控制器输出的控制信号作用下， R_3 寄存器的内容被送到地址形成部件，生成地址信号输出到地址总线，再经由译码器译码后，寻址源操作数存放的内存单元，操作控制器发出读信号，将源操作数读到数据总线，并加载到 R_1 寄存器。第二条指令执行完毕。

2.14 假设 A 和 B 是同一条总线所连接的两个存储器单元，总线位宽大于或等于存储单元的位数。现在需要将 A 单元的内容传送到 B 单元中，能否在一个总线周期内完成传送任务？为什么？

不能。

存储器单元之间不能直接传送数据，必须先将源操作数从存储器中读出到某个寄存器暂存，再将暂存的内容写入目的存储单元。（参考讲义 2.5.1 的 Move 操作）

2.15 假设 I_j 和 I_{j+1} 是前后相继的两条指令，请举例说明指令流水线的“WAR”和“WAW”两种数据相关问题。

举例说明，比较结果有何不同即可，这里不要求明确具体为何流水线会出现“WAR”和“WAW”

WAR 为“读后写”，假设 I_j 为 Sub R_1, R_2, R_3 ， I_{j+1} 为 Move R_2, R_4 ，若在 sub 指令读取 R_2 之前 move 就将 R_4 中数据转移到了 R_2 ，则会发生“WAR”。

WAW 为“写后写”，假设 I_j 为 Add R_1, R_2, R_3 ， I_{j+1} 为 Move R_1, R_3 ，若 move 先将 R_3 中数据转移到了 R_1 ，则最终 R_1 存放的是 $R_2 + R_1$ 的结果，而程序本意是保留 R_3 转移到 R_1 的数据。

2.16 名称解释：（1）转移目标指令；（2）转移代价；*（3）转移延迟槽；*（4）BTB。此题*标识的（3）和（4）不是作业。

转移目标指令：一般情况下，CPU按顺序执行指令，但遇到转移指令时，会跳转到转移目标，开始执行转移目标地址中的指令，该指令就是转移目标指令。

转移代价：由于转移指令产生的流水线周期延迟被称为转移代价。

2.20 什么是同构多核与异构多核？采用异构多核的目的是什么，试举例说明。

讲义 2.5.5

1. 同构多核处理器

同构多核处理器的内核普遍采用通用处理器，每个处理器的结构相同，地位相等。同构多核的结构相对简单，硬件实现复杂度低。

2. 异构多核处理器

异构多核处理器通过配置具有不同功能和性能的内核以匹配实际应用需求，在提升芯片整体性能的同时，优化处理器结构，降低系统功耗。

例如，对于通用的个人计算机，可以将图形处理器 GPU（Graphic Processing Unit）与通用 CPU 集成在一颗芯片上，从而构成一种异构多核处理机。在这样的架构下，程序中必须串行执行的部分由通用 CPU 上执行，可以并行的（例如视频和图形图像处理）处理任务交由 GPU 内核进行提速。

2.25 作为一种性能指标，MIPS 是否能客观反映计算机的运算速度？为什么？

不能，例如执行同样的任务，RISC 计算机相比 CISC 计算机要花费更多的指令。

不够客观。原因有二：

- 1.考虑到 RISC 与 CISC 架构处理器指令集繁简程度不同，使用 MIPS 衡量二者间性能差异不妥。
- 2.对于同一处理器，MIPS 还依赖于所运行的测试程序。

3.5 什么是主存储器？什么是外存储器？

主存储器，简称内存，是计算机运行过程中的存储主力。用于存储指令(编译好的代码段)，运行中的各个静态、动态、临时变量，外部文件的指针等。

外存储器，是用于存储需要永久存储文件的机械硬盘、固态硬盘以及便携式存储器。

3.6 辅助存储器常见的几种接口标准？

共7种，IDE接口、SCSI接口、SATA接口、SAS接口、SD接口、eMMC接口和UFS接口

3.8 EPROM、EEPROM、Flash的共同特点是什么？这些存储器具有读写功能，为什么仍是ROM？

共同特点：可重复擦写、非易失性、工作原理均基于对MOS管电荷的修改。

ROM仅表示掉电后信息不会丢失，只读是相对CPU的读写控制电平及其控制逻辑而言，仍称为ROM只是一种习惯

3.10从DDR到DDR4有哪些改进？ 关键技术的改进：

关键技术的改进：

1. Bank Group架构。使用2或4个可选的采用8n预取的Bank Group分组，每个Bank Group可以独立读写数据
2. 点到点传输。每个通道只连接一根内存条，消除了共享传输带来的性能瓶颈
3. 3D堆叠。在散热允许的情况下，采用3D堆叠封装技术，使得单根内存条容量从目前的8GB提高到64GB

其他技术变化

1. DDR4功耗明显降低，传输速度明显加快
2. DDR4增加了DBI、CRC、CA parity功能，增强了信号的完整性、改善数据传输及存储的可靠性

3.12 虚拟存储器解决了哪些问题？

虚拟存储器主要解决了物理空间受限不能运行更大的系统程序的问题。

3.16 Cache的工作原理、作用是什么？在那些场合使用Cache？

Cache的工作原理：Cache中存放近期需要重复运行的指令数据，形成主存储器内容的副本。

1. 主存和Cache都会采用分字块的方式进行管理，Cache中保存的就是对应的主存字块的一个副本。每一个Cache字块都会有一个标记位，用于表示当前字块里存放的是哪一个内存字块的副本。通过这个标准位，CPU就可以判断出希望访问的内存字块是否已经存在于Cache中；
2. 当Cache已经用满，但主存还需将新的字块调入Cache时，就会执行一次Cache字块的替换。替换的规则称为替换策略或替换算法，由替换部件执行；
3. 通过两种常用的写入方式：一是先写Cache字块，待Cache字块被替换出去时再一次性写入内存字块；再一个是在写Cache字块的同时也写入内存字块，来保证Cache字块和内存字块的一致性。

Cache的作用：解决CPU和经常访问的主存储器速度差距，尽量减少CPU访问DRAM的频率，在保证系统性能的前提下，降低存储器系统的实现代价。

在哪些场合下使用Cache：CPU与主存之间、显示系统、硬盘和光驱，以及网络通讯中，都需要使用Cache技术。



3.18 Cache line应该含有那些基本信息项?

Cache 的基本单元称为行 (Line, Cache line) 或字块或块。每个字块应包括的基本信息如下。有①数据字段: 保存从主存单元复制过来的数据, 单位是 (区) 块。每个区块的大小一般介于 4~128 字节之间, 典型大小为 32 或 64 字节。②标记字段: 保存数据字段在主存中的地址信息, 又称为地址标记寄存器, 记为 Tag。③有效位字段: 指示区块和 Tag 是否有效。

3.19 Cache常用的替换策略有哪些?

随机替换 (Random) 策略、最不经常使用 (LFU) 替换策略、先进先出 (FIFO) 策略、近期最少使用 (LRU) 替换策略



3.20 某计算机按字节编址，其主存容量为1MB，Cache容量为16KB，Cache和主存之间交换的块大小为64B，采用直接相联映射方式。

(1) Cache共有多少个字块 (Cache line)？ (2) 主存地址为02021H的单元装入Cache后对应的Cache地址是？ (3) 主存地址为02021H的单元装入Cache后存放在Cache中的第几字块中 (Cache起始字块为第0字块)？

T (6位)、C (8位)、W (6位) 0000 00,10 0000 00,10 0001

(1) 256个字块 (16KB/64B)

(2) 10 0000 00 0000 00,10 0000 00,10 0001

(3) 128字块 0000 00,10 0000 00,10 0001



3.21 某计算机按字节编址，其主存容量为1MB，Cache容量为16KB，Cache和主存之间交换的块大小为64B，采用8路组相联映射方式。

(1) 主存地址中页号s、页内块号u、块内地址w各占多少位？ (2) 主存地址为02021H的单元装入Cache后存放在Cache中的第几组 (起始组为第0组)？ (3) Cache line对应的Tag字段占用多少位？

(1) 页号s:9位、块号u:5位、地址w:6位

(2) 第0组 0000 0010 0,000 00,10 0001

(3) 9位

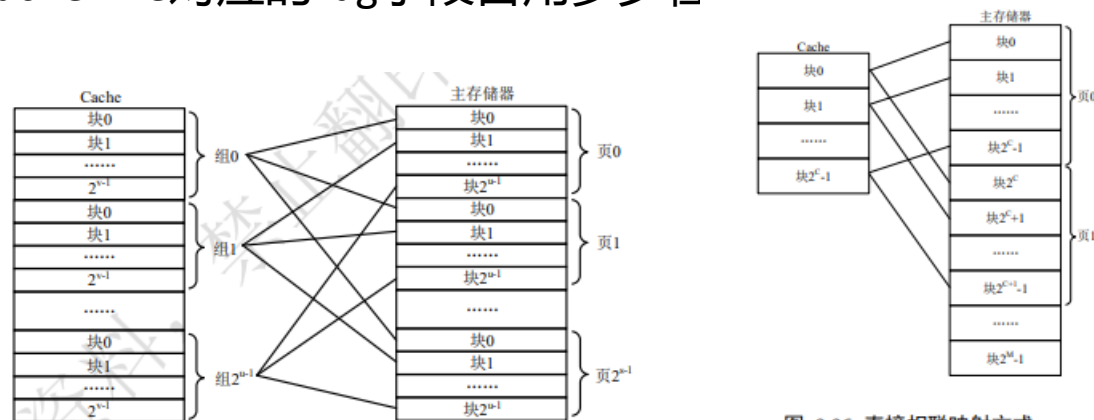


图 3.38 组相联的映射关系

图 3.36 直接相联映射方式



3.22 某按字节编址的计算机系统，使用了40位地址线，16位数据线，请问该计算机存储器空间的寻址范围是什么？

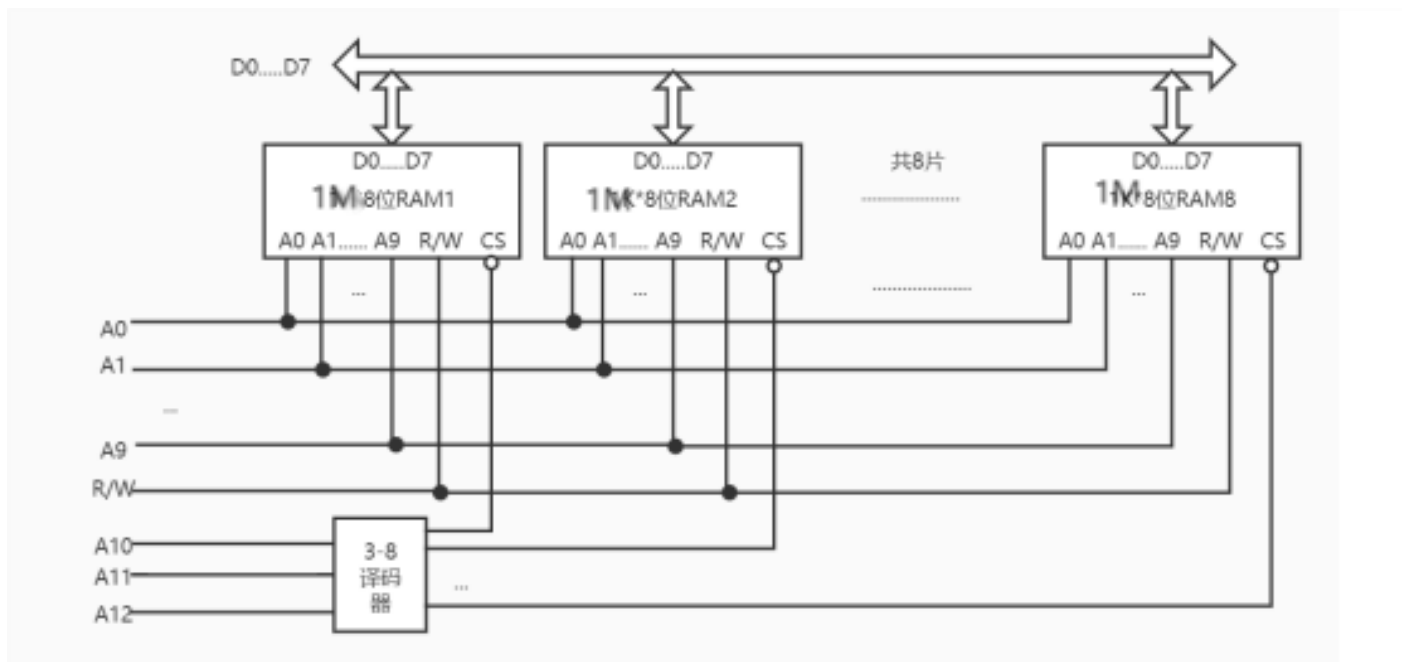
$$2^{40} \times 8 = 2^{43}$$

$$2^{43} / 8 = 2^{40}$$

寻址范围是：0x000000000000~0xFFFFFFFFFFFF (10个0， 10个F)



3.23 试用1M×1位的芯片构成1M×8位的存储器。3.24， 3.27同理



3.28 NOR与NAND Flash在嵌入式系统中如何连接？功能有何异同？

NOR flash各基本存储单元Bit line是并联的，而NAND Flash是串联的

NOR flash可按位随机读取，但写入和擦除速度较慢；

NAND Flash随机读取速度慢且不能按字节随机编程，写入和擦除速度较快

第 4 章作业

第一部分

4.2 举例说明何为总线复用？

4.1.1 节

按照是否复用，总线可分为时分复用总线和非复用总线。

5) 按时分复用方式分类

在许多系统中，为了减少物理信号线或者芯片引脚数量，往往将不同的信号共用一条或者一套物理信号线或者芯片引脚。这些物理信号线或者引脚在不同的时刻体现不同的功能和用途。例如，在 Intel 8086 芯片上，20 位地址总线的低 16 位与 16 位数据总线共用物理引脚。在总线周期的最初时钟周期，这些物理引脚上出现的信号表示地址，在接下来的时钟周期这些引脚又用来传输数据。这种方式称作时分复用。在系统总线上，如果需要，可以使用锁存器将分时复用的信号进行分离。总线中也有一些信号无需分离，在不同的时钟周期表示不同的信号。例如，PCI 总线中的 $\overline{C/BE}[3:0]$ 在传输地址周期表示总线命令；在传输数据周期，这 4 条信号线用于字节使能（指明 32 位总线上哪些字节有效）。

一般来说，采用非时分复用总线有利于提高数据传输速率，而采用时分复用总线有利于减少芯片引脚或者物理信号线的数量，设计师需要在两者之间寻求平衡。

4.4 某计算机系统的地址总线宽度是 13 位，其数据总线宽度是 11 位，再不采用总线分时复用的情形下，请计算该计算机的最大存储器空间寻址范围。

13 位地址总线可寻址 2^{13} 个存储单元

数据总线宽度为 11 位，则可寻址 $2^{11} \times 11 = 90112 \text{b} = 90.112 \text{kb} = 11 \text{kB}$

补充：kb 和 KB

比特	计算机中数据量的单位，也是信息论中信息量的单位。一个比特就是二进制数字中的一个1或0。	速率	连接在计算机网络上的主机在数字信道上传送比特的速率，也称为比特率或数据率。
	常用数据量单位		常用数据率单位
	$8 \text{ bit} = 1 \text{ Byte}$		$\text{bit/s} \text{ (b/s, bps)}$
	$\text{KB} = 2^{10} \text{ B}$		$\text{kb/s} = 10^3 \text{ b/s (bps)}$
	$\text{MB} = \text{K} \cdot \text{KB} = 2^{10} \cdot 2^{10} \text{ B} = 2^{20} \text{ B}$		$\text{Mb/s} = \text{k} \cdot \text{kb/s} = 10^3 \cdot 10^3 \text{ b/s} = 10^6 \text{ b/s (bps)}$
	$\text{GB} = \text{K} \cdot \text{MB} = 2^{10} \cdot 2^{20} \text{ B} = 2^{30} \text{ B}$		$\text{Gb/s} = \text{k} \cdot \text{Mb/s} = 10^3 \cdot 10^6 \text{ b/s} = 10^9 \text{ b/s (bps)}$
	$\text{TB} = \text{K} \cdot \text{GB} = 2^{10} \cdot 2^{30} \text{ B} = 2^{40} \text{ B}$		$\text{Tb/s} = \text{k} \cdot \text{Gb/s} = 10^3 \cdot 10^9 \text{ b/s} = 10^{12} \text{ b/s (bps)}$

例1：有一个待发送的数据块，大小为100 MB，网卡的发送速率为100 Mbps，则网卡发送完该数据块需要多长时间？

$$\frac{\cancel{100} \text{ MB}}{\cancel{100} \text{ Mb/s}} = \frac{\text{MB}}{\text{Mb/s}} = \frac{2^{20} \text{ B}}{10^6 \text{ b/s}} = \frac{2^{20} \cdot 8 \text{ b}}{10^6 \text{ b/s}} = 8.388608 \text{ s}$$

严格来说，不能直接约掉

4.5 计算机系统中总线层次化结构是怎样的？请给出典型的 PC 总线示意图和典型基于 AMBA2 的 MCU 系统的总线示意图。

答案不唯一，典型的 PC 总线示意图体现出多总线结构即可，典型基于 AMBA2 的 MCU 系统的总线体现出互连结构即可

总线的层次结构：在同一计算机系统内往往同时存在片内总线、系统总线和外总线，大部分计算机系统采用多级分层总线结构，如图 4.1 所示。

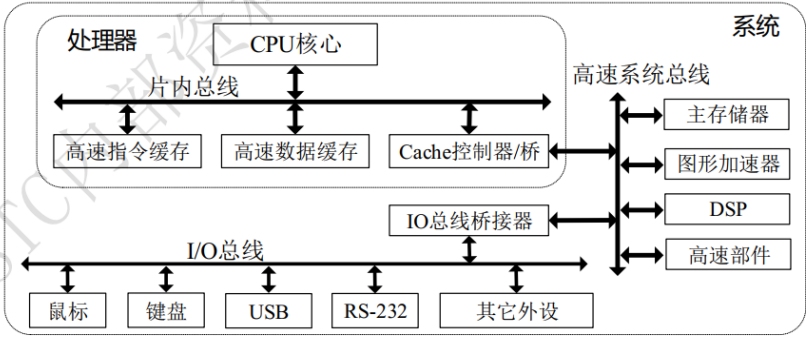


图 4.1 总线的层次结构

四总线结构：PC 机中一度广泛使用的 PCI 总线就属于 4 总线结构。在 PCI 总线结构中，按照地图上的方位，图 4.8 上方的 Cache/桥又被称为北桥，而下方的扩展总线接口被称为南桥。

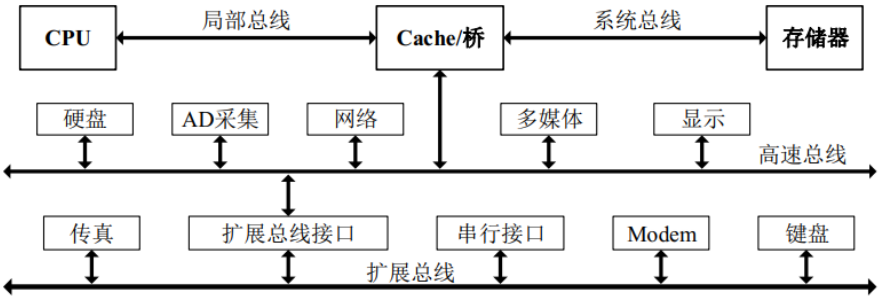


图 4.8 典型的四总线结构

AMBA2 总线的典型互连结构如图 4.18 所示。AMBA2 总线基于一条高性能系统中枢总线（可采用 AHB 或 ASB），以实现 CPU、片上存储器、外部存储器及 DMA 设备之间的高速数据传输。这条高性能总线有一个桥接器，用来连接低功耗低速率的 APB，并由 APB 连接外设。

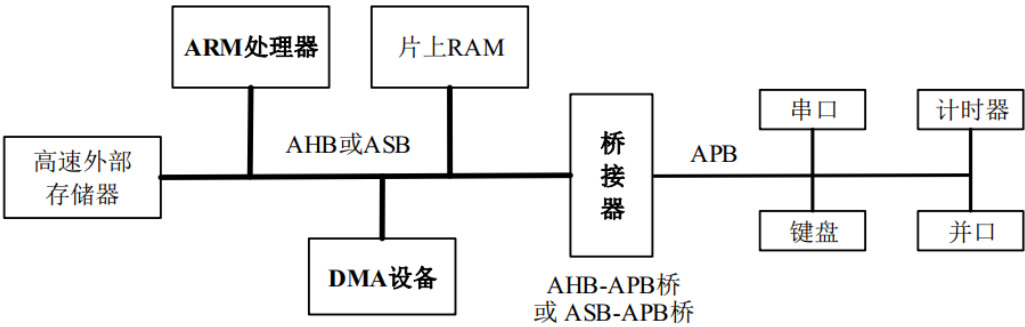


图 4.18 典型 AMBA2 总线的微控制器系统

补充：MCU（来源：百度百科）

微控制单元 (Microcontroller Unit；MCU) ，又称单片微型计算机 (Single Chip

Microcomputer) 或者单片机, 是把中央处理器(Central Process Unit; CPU)的频率与规格做适当缩减, 并将内存(memory)、计数器(Timer)、USB、A/D 转换、UART、PLC、DMA 等周边接口, 甚至 LCD 驱动电路都整合在单一芯片上, 形成芯片级的计算机, 为不同的应用场合做不同组合控制。诸如手机、PC 外围、遥控器, 至汽车电子、工业上的步进马达、机器手臂的控制等, 都可见到 MCU 的身影。

4.8 计算机系统什么情况下需要总线仲裁 (arbitration) ?

4.1.2 节

总线仲裁 (bus arbitration) 就是在多总线主设备的环境中提出来的。总线仲裁又称总线判决, 其目的是合理地控制和管理系统中多个主设备的总线使用请求, 以避免冲突。

4.13 异步总线有哪些可能的握手方式?

不互锁, 半互锁, 全互锁

4.15 周期分裂式总线操作时序有哪些特点? 适用于什么样的场景?

为了提高系统的总体性能, 出现了周期分裂式总线时序。以读存储器操作为例, 整个总线周期可以分解为两个子周期: 寻址子周期和数据传送子周期。在寻址子周期, 主模块发送地址、读命令和主模块识别码 (主模块 ID 编号), 由被寻址的从模块接收。随后, 主模块释放总线, 以便总线可以被其他主模块使用。待从模块准备好数据之后, 由从模块发起总线使用申请, 获准后启动数据传输子周期, 从模块输出数据和主模块识别码, 相关主模块接收数据。这种将总线周期进行分解的模式, 减少了总线资源的无效占用, 从而提高了总线的利用率。

适用场景: 应用于多主模块系统, 减少总线资源的无效占用

4.19 为什么 AMBA 总线中没有定义电气特性和机械特性?

AMBA总线是片内总线, 是一种与工艺无关的片上协议, 所以无需指明通信实体间硬件连接接口的机械特点。电气特性取决于设计时选择的生产工艺。

4.21 简述 AHB 总线的流水线机制。

实际上, 传输过程中地址信息的更新和数据的更新在节拍上是错开的, 当前 AHB 传输地址阶段的地址信息实际上是上一次 AHB 传输最后一个时钟周期就已经被驱动到 HADDR[31:0] 上了, 而本次 AHB 传输的数据更新至 HRDATA[31:0] (读传输) 之后, 在下次 AHB 传输开始的第一个时钟周期才被读取。这种地址信息和数据信息交叠 (overlapping) 的操作方式, 被称为流水线 (pipeline) 机制。

4.22 简析 AHB 中 SPLIT 操作的优点。

表 4.4 中 HMASTER[3:0]、HMASTLOCK、HSPLIT[15:0]信号用于支持名为“SPLIT”的传输模式。这种传输模式在 ARM 公司的资料中，被称作分裂式操作（Split transactions）；其它一些资料中也称作流水线分离。

SPLIT 传输，指 AHB 传输的两个阶段分离（地址阶段和数据阶段可以被分离，参见 4.1.4 小节）。根据前述 AHB “流水线”机制可知，当前 AHB 传输的数据实际上在下一次 AHB 传输的一开始才被读取的，第 n 次 AHB 传输的地址在第 $n-1$ 次 AHB 传输的时候就已经被驱动到了地址总线上。这样“驱动地址”和“驱动数据”两个操作构成了两级流水线操作。

这种两级的流水线操作要求从机能够及时地响应主机，在主机驱动地址到地址总线 HADDR[31:0]后，能够被从机及时读取；主机驱动数据到数据总线 HWDATA[31:0]上后，也要能够被从机及时读取。如果从机因为某种原因不能及时响应，这个流水线就会被打断，影响到总体性能。为了应对这种从机不能及时响应的情形，出现了流水线分离设计。

4.23 解释图 4.23 中 HREADY 信号的作用。

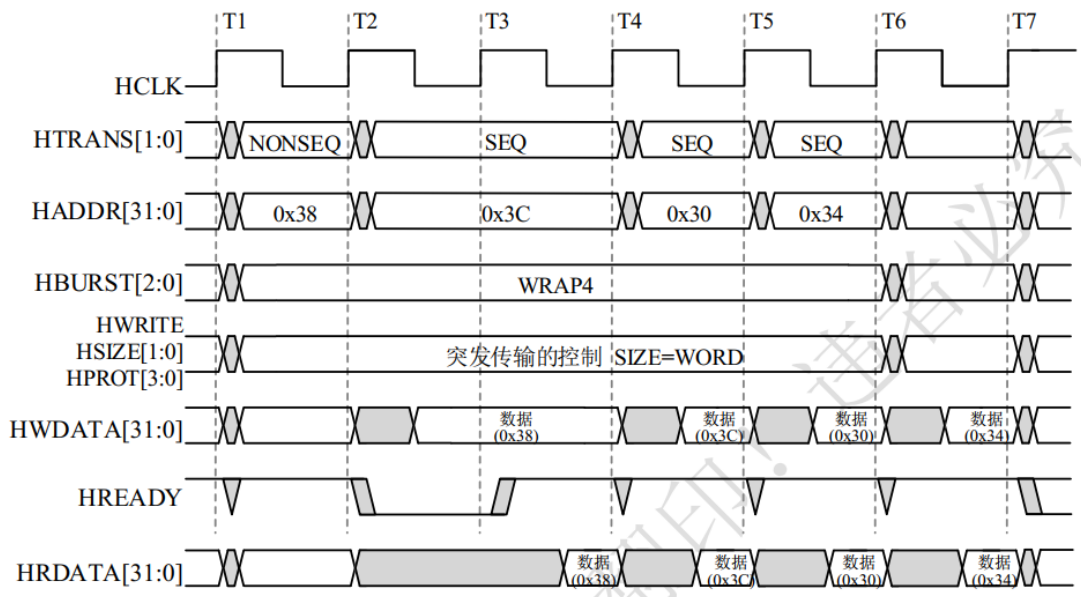


图 4.23 WRAP4（Four-beat wrapping burst）突发传输时序

直接按书上写 HREADY 可能被拉低，不够准确，需要针对问题中的图具体说明，答案参考下面横线部分。

以图 4.21 中读传输为例，主机在 HCLK 上升沿将地址和控制信号驱动到总线上。在时钟的下一个上升沿，从机采样地址和控制信号，随后进入数据阶段。因从机在数据阶段的第一个时钟周期没能准备好，故把 HREADY 拉为低电平，从而插入了等待周期（图 4.21 中从机插入了两个时钟周期的等待）。从机准备好后，再将 HREADY 重新置为高电平，并将数据驱动至 HRDATA[31:0]。主机在随后的下一个时钟（第五个时钟）上升沿采样 HRDATA[31:0]获得从机响应的数据。读传输和写传输过程是有差异的：在写操作过程中，主机必须确保数据在整个扩展期内稳定；而在读传输过程中，从机不必一直确保数据有效，只需要在相应周期提供数据即可。

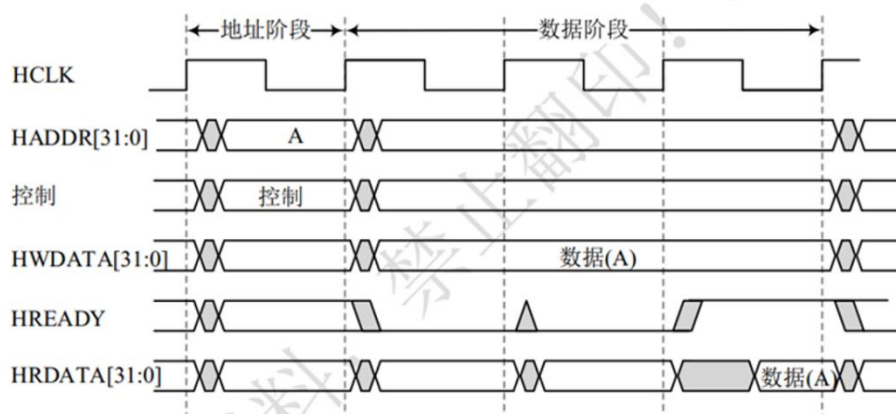


图 4.21 有等待状态的 AHB 传输时序

补充：AMBA 规范里的规定

2.1.1 AHB signal prefixes

H indicates an AHB signal.

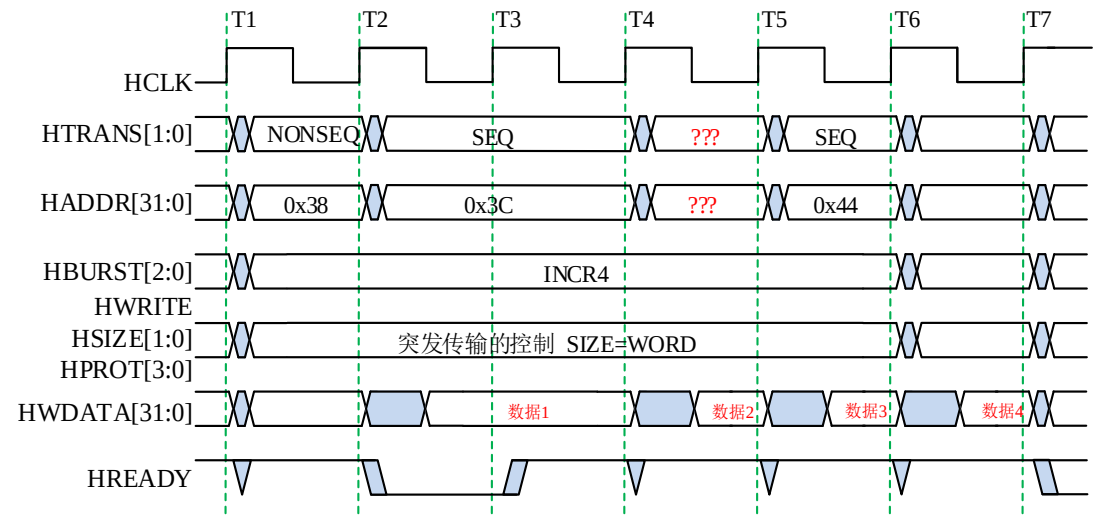
For example, **HREADY** is the signal used to indicate that the data portion of an AHB transfer can complete. It is active HIGH.

4.25 AHB 中突发传输定义了哪些类型？各自有什么特点？

表 4.5 AMBA2 中突发传输类型信号 HBURST[2:0]含义

HBURST[2:0]	类型	类型的描述
000	SINGLE	单次传输 Single transfer
001	INCR	未标识长度的地址递增式传输 Incrementing burst of unspecified length
010	WRAP4	突发长度为 4 的地址循环递增式传输 4-beat wrapping burst
011	INCR4	突发长度为 4 的地址顺序递增式传输 4-beat incrementing burst
100	WRAP8	突发长度为 8 的地址循环递增式传输 8-beat wrapping burst
101	INCR8	突发长度为 8 的地址顺序递增式传输 8-beat incrementing burst
110	WRAP16	突发长度为 16 的地址循环递增式传输 16-beat wrapping burst
111	INCR16	突发长度为 16 的地址顺序递增式传输 16-beat incrementing burst

4.62 下图中 HTRANS[1:0]信号“???”应该为什么取值？HADDR[31:0]信号“???”应该为什么取值？HWDATA[31:0]中数据 1、数据 2、数据 3、数据 4 对应的地址分别是什么？



SEQ
0x40
0x38,0x3C, 0x40,0x44

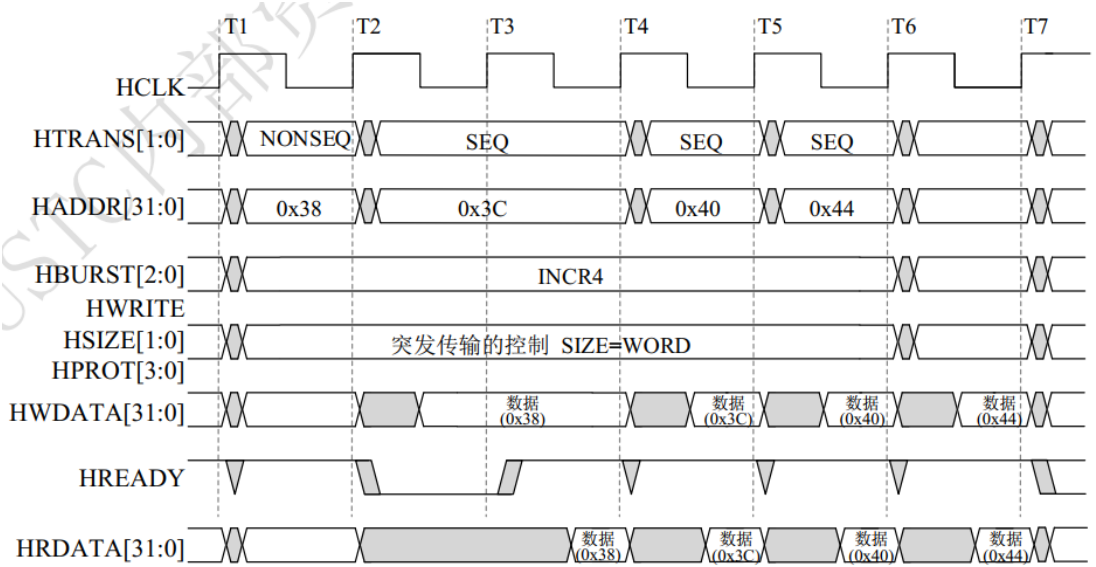
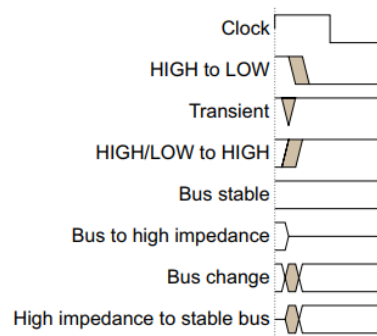


图 4.24 INC4（Four-beat incrementing burst）突发传输时序

补充：

Timing diagrams

The figure named *Key to timing diagram conventions* explains the components used in timing diagrams. Variations, when they occur, have clear labels. You must not assume any timing information that is not explicit in the diagrams. Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.



Key to timing diagram conventions

Single-bit signals are sometimes shown as HIGH and LOW at the same time and they look similar to the bus change shown in *Key to timing diagram conventions*. If a single-bit signal is shown like this then its value does not affect the accompanying description.

第二部分

4.30 PCIe 5.0 版本中 X16 的吞吐量 63.0 GB/s 是如何计算得到的？

5.0-1.0-PUB — PCI Express® Base Specification Revision 5.0 Version 1.0

The primary Link attributes for PCI Express Link are:

- The basic Link - PCI Express Link consists of dual unidirectional differential Links, implemented as a Transmit pair and a Receive pair. A data clock is embedded using an encoding scheme (see Chapter 4) to achieve very high data rates.
- Signaling rate - Once initialized, each Link must only operate at one of the supported signaling levels.
 - For the first generation of PCI Express technology, there is only one signaling rate defined, which provides an effective 2.5 Gigabits/second/Lane/direction of raw bandwidth.
 - The second generation provides an effective 5.0 Gigabits/second/Lane/direction of raw bandwidth.
 - The third generation provides an effective 8.0 Gigabits/second/Lane/direction of raw bandwidth.
 - The fourth generation provides an effective 16.0 Gigabits/second/Lane/direction of raw bandwidth.
 - The fifth generation provides an effective 32.0 Gigabits/second/Lane/direction of raw bandwidth.
- Lanes - A Link must support at least one Lane - each Lane represents a set of differential signal pairs (one pair for transmission, one pair for reception). To scale bandwidth, a Link may aggregate multiple Lanes denoted by xN where N may be any of the supported Link widths. A x8 Link operating at the 2.5 GT/s data rate represents an aggregate bandwidth of 20 Gigabits/second of raw bandwidth in each direction. This specification describes operations for x1, x2, x4, x8, x12, x16, and x32 Lane widths.
- Initialization - During hardware initialization, each PCI Express Link is set up following a negotiation of Lane widths and frequency of operation by the two agents at each end of the Link. No firmware or operating system software is involved.
- Symmetry - Each Link must support a symmetric number of Lanes in each direction, i.e., a x16 Link indicates there are 16 differential signal pairs in each direction.

表 4.14 PCI-E 不同版本的传输速率

PCI-E 版本	发布时间	编码方式	传输速率				
			×1	×2	×4	×8	×16
1.0	2003	8b/10b	250 MB/s	0.50 GB/s	1.0 GB/s	2.0 GB/s	4.0 GB/s
2.0	2007	8b/10b	500 MB/s	1.0 GB/s	2.0 GB/s	4.0 GB/s	8.0 GB/s
3.0	2010	128b/130b	984.6 MB/s	1.97 GB/s	3.94 GB/s	7.88 GB/s	15.8 GB/s
4.0	2017	128b/130b	1969 MB/s	3.94 GB/s	7.88 GB/s	15.75 GB/s	31.5 GB/s
5.0	2019	128b/130b	3938 MB/s	7.88 GB/s	15.75 GB/s	31.51 GB/s	63.0 GB/s

吞吐量 = 单个通道的传输速率 (Gb/s/lane) * 编码率 (b/b) * 通道数目 (lane)
32Gb/s/lane * 128b/130b * 16=63.0Gb/s (用到的数据在上图用红框标出)

4.35 异步串行通信中的起始位和停止位有什么作用？

1) 异步串行数据帧格式

异步串行通信方式在传输数据时，待传输的数据以字符（一个字符通常含五至八位）为单位进行传送，传输字符也常被称为数据帧。为了能够区分数据帧的起和止，发送方在每个需要传送字符之前增加一个起始位，在每个字符之后增加一个停止位。

4.38 什么是 I/O 端口？一般接口电路中有哪些端口？

注意区分 IO 端口和 IO 接口电路，计算机系统中可能有多多个 I/O 接口，每个 I/O 接口可以有多个 I/O 端口。

通常 I/O 接口电路用一组寄存器来存储这三类信息，CPU 通过地址对这些寄存器进行访问。这些寄存器称作端口寄存器或 I/O 端口，简称端口（Port）。CPU 通过读/写端口寄存器实现与外设的数据交换。对应上述三类信息，接口电路中端口寄存器也分为三类：①数据端口（寄存器）：用来存放 CPU 和外设间交换的数据，具有输入和输出数据缓冲功能。②状态端口（寄存器）：用来存放外设状态，如准备就绪（Ready）、忙碌（Busy）和错误（Error）等。③控制端口（寄存器）：用来存放 CPU 发往外设的控制命令，如启动、停止和复位等。

4.39 CPU 对 I/O 端口的编址方式有哪几种？各有什么特点？

1) 内存映像编址

内存映像编址（memory mapped I/O addressing）也称作 I/O 端口统一编址。这种编址方式将 I/O 接口中的每个端口视为主存的存储单元，每个端口占用一个存储单元地址，把主存地址空间的一部分划出来用作 I/O 地址空间。如摩托罗拉公司 68 系列、Intel 公司 51 系列、ARM 和 PowerPC 等处理器，就采用 I/O 端口统一编址。内存映像编址也称为“I/O 内存”方式，I/O 寄存器位于“内存空间”。

内存映像编址时，访问 I/O 寄存器与访问内存使用相同的指令，由于访问存储器的指令功能相对较强，编程时使用访问存储器指令读写 I/O 端口，操作更加灵活，程序更加简洁。这种方式的缺点是占用了存储器空间；此外在阅读程序时，只能依据地址来区分是访问内存还是访问 I/O，程序可读性不好。

2) I/O 端口独立编址

I/O 端口独立编址又称为分离编址（Isolated I/O Addressing），即存储器和 I/O 端口位于两个相互独立的地址空间，存储器和 I/O 端口独立编址。独立编址方式与前述统一编址方式相反，所有端口地址单独构成一个地址空间，I/O 地址空间与存储器地址空间互不相干。例如，80x86 系列微处理器就采用 I/O 端口独立编址方式。

这种编址方式需要设计单独的 I/O 指令，专门用于访问 I/O 端口，还需要使用单独的信号线，指示当前的总线周期是访问内存还是访问 I/O。

4.40 接口电路的输入需要用缓冲器，而输出需要用锁存器。为什么？

缓冲器：使高速工作的 CPU 与慢速工作的外设起协调和缓冲作用，实现数据传送的同步。

锁存器：由于 CPU 速度高于外设速度，一般使用锁存器来保持 CPU 输出数据，供外设读取。

4.41 CPU 与 I/O 设备之间的数据传送有哪几种方式？每种工作方式的特点是什么？ 这四种方式都是 CPU 与外设之间的数据传送方式，具体特点可以自行总结

如前所述，外设的多样性使 I/O 接口电路的差异很大，CPU 与外设接口之间数据传送方式也有多种多样，常用的有无条件传送方式、状态查询传送方式、中断传送方式和 DMA 传送方式等。

1. 无条件传送方式

某些外设工作时始终处于“准备好”的状态，如按键和 LED 字符显示器等。这些外设随时能够接受 CPU 的访问，处理器不必关心其状态，故此类外设常称为无条件外设。CPU 对无条件外设进行输入输出操作时不需要考虑其状态，此类访问被称为无条件传送方式。

2. 查询传送方式

事实上，很多外设在对其进行操作之前需要先确认其工作状态，只有状态许可时才可以访问。例如，A/D 转换器转换结束后才能读转换结果；CPU 向打印机输出数据时，必须打印机准备好之后方可进行。此类须满足一定条件才能访问的外设称为条件访问外设，绝大多数计算机外设都属于条件访问外设。CPU 对此类外设进行输入或输出操作时需要先查询外设的状态，对此类外设的访问方式称为条件 I/O 传送方式。

条件 I/O 传送有两种主要实现方式：查询传送方式和中断控制方式。

查询传送方式又称为程序查询方式，或简称查询方式。其原理和过程为：CPU 访问数据端口前，先查询该设备的状态；设备“准备好”时，CPU 才对数据端口进行输入/输出。为了实现状态的查询，接口电路中既要有数据端口，又要有状态端口。状态查询方式 I/O 接口电路原理如图 4.47 所示。

3. 中断传送方式

状态查询方式中，CPU 需要反复执行大量外设状态查询指令，故而 CPU 利用率不高。在中断传输方式中，平时 CPU 执行正常的处理任务，当外设状态改变或者需要与 CPU 进行数据交换时，由 I/O 接口主动向 CPU 发出数据传送请求，CPU 中断当前的任务执行，转而执行相应的中断服务程序，为外设（I/O 接口）进行服务。

中断传送方式通常会引入专门的中断控制器，对多个 I/O 接口的中断请求进行管理，中断控制器可以是独立电路单元（例如 Intel 8259A 可编程中断控制器），也可以与 CPU 集成在一个芯片上（如 ARM Cortex-M3/M4 处理器中的 NVIC）。中断传送的大致过程可以图 4.49 所示中断传输方式接口电路为例，示意图。

当 I/O 接口需要对其进行服务时，I/O 接口向中断控制器发出中断请求 IRQ（Interrupt Request）；中断控制器收到某个 IRQ 之后，对其中断条件以及中断优先级进行检查，如果符合条件，中断控制器向 CPU 发出外部中断请求信号 INT（Interrupt）。CPU 收到 INT 后暂停当前的程序执行，转去执行中断服务程序，在中断服务程序中为 I/O 接口进行服务（如读/写数据等）；中断服务程序执行完毕之后，CPU 返回原来被中断的程序断点，继续执行被中断的程序。

4.DMA 传送方式

DMA 方式顾名思义是在内存与 I/O 接口之间直接进行传输，无需 CPU 干预。为此，DMA 传送方式引入了一个专用电路单元，其功能是与 CPU 协商并获得系统总线控制权之后，在 I/O 接口和存储器之间建立一条可直接传输数据的临时通道，通过地址总线和专用引脚信号同时选中需要进行数据传送的内存单元和 I/O 端口，在一个总线周期内先后发读信号和写信号，将需要传送的数据读出至数据总线并写入目的单元，在一个总线周期内完成一次数据传送。这个专用电路单元被称为 DMAC（Direct Memory Access Controller，DMA 控制器）。

4.44 分析图 4.55 所示菊花链电路，某计算机系统有 4 个中断源，设计基于菊花链的优先级排队电路（画出电路示意图），并指出优先级最高的是哪个中断源。

把图里的 8 个中断源改成 4 个即可，优先级最高的为最靠近链前端（即靠近 CPU）的中断源 1

在图 4.55 中，假设中断请求信号和中断响应信号 INTA 都是高电平有效。CPU 送出 INTA 后，若设备 1 有中断请求，则与门 A1 开，与门 A2 关，中断响应信号送至设备 1 而不再向下一级传送。反之，若设备 1 无中断请求，则与门 A1 关，与门 A2 开，中断响应信号继续向下一级传送；若设备 2 有中断请求，则与门 B1 开，中断响应信号送至设备 2，同时与门 B2 关，中断响应信号不再向下传送（无论后面的设备是否有中断请求）。依据上述分析，设备接入菊花链的顺序决定了设备的中断优先级：越靠近链前端（靠近 CPU）的设备优先级越高。

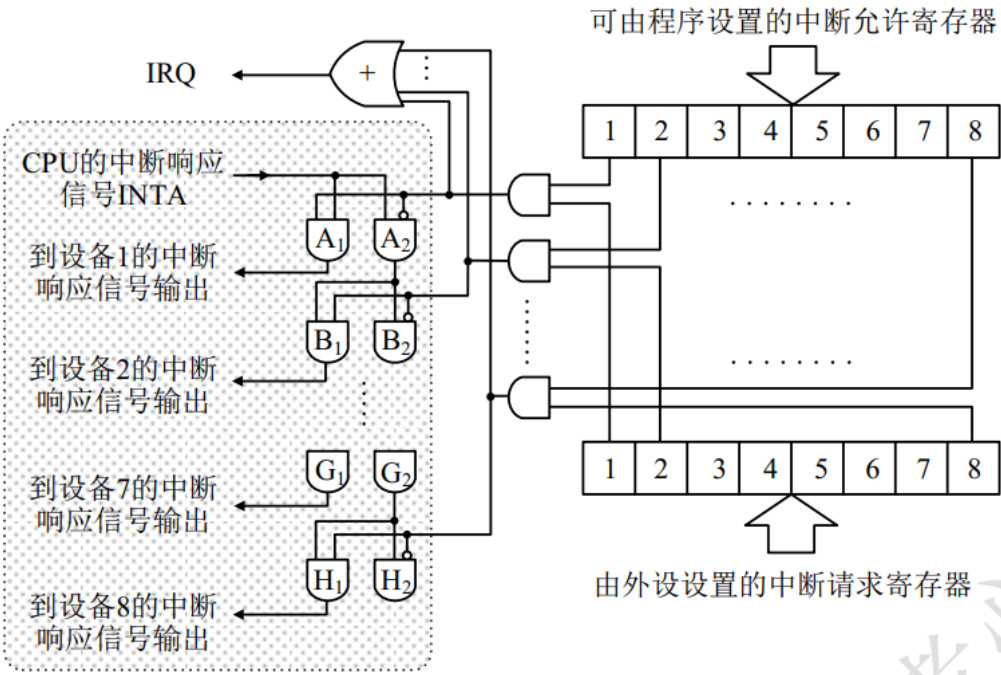


图 4.55 菊花链式优先权排队电路

4.45 名词解释：中断向量表。

2) 中断向量

8086 和 ARM 等处理器中，中断向量是中断服务子程序的入口地址。通常在处理器设计中，把所有中断服务子程序入口地址集中存放在存储器的特定区域（如 8086 系统中是内存的最低 1KB），这个特定的区域称为 IVT（Interrupt Vector Table，中断向量表⁵）。中断发生时，CPU 根据中断类型码到 IVT 进行查询，找到对应的中断服务程序入口地址。

4.46 数据块传送方式的 DMA 适用于什么场景？

题里问的是数据块传送方式的 DMA，数据块传送方式又叫成组传送方式

上述成组传送方式适合大数据量的快速传送，但是 DMAC 长时间占用总线将使得 CPU 无法处理某些紧急事务，因此成组传送方式只能在特定条件下使用。询问传送方式非常适合与数据通信接口芯片之间进行数据交换。

4.51 什么是矩阵键盘的行扫描法？

总结出大致流程即可

行扫描法首先判断是否有键按下。方法是先在输出端口的各位输出“0”（即低电平），再从输入端口读取数据，如果读取的数据是 1111 1111B，则说明当前所有行线处于高电平状态，没有键被按下。如果并行输入端口读取的数据不是 1111 1111B，则说明必有行线处于低电平，也就是说肯定有键被按下。

在确定有按键被按下后，行扫描法按照图 4.65 所示的流程确定哪个键被按下。基本思路是：逐列检查是否有按键被按下，发现有按键被按下后再确定行（此即行扫描，对应图 4.65 中阴影区域部分）。为方便描述，定义图 4.64 (b)所示 8×8 矩阵键盘中按键的键号为（列号×8+行号）。

逐列检查过程：程序先将键号置零，将计数值设为键盘列的数目，然后再设置扫描初值。首先设置扫描初值为 1111 1110B，即使第 0 列为低电平，而其他列为高电平。输出扫描初值后，马上读取行线的值，看是否有行线处于低电平。如果没有表示该列没有按键按下，将扫描初值循环左移一位，变成 1111 1101B，即使第 1 列为低电平，其他列为高电平。这样进入第 1 列的扫描，使键号为 8（第 1 列的第 0 号键），计数值减 1。如此循环，直到计数值为 0 结束。

如果在逐列检查过程中，发现有行线为低电平，则表示此列有按键被按下。例如，读入的行值为 1111 1011B 表示按下的按键在第 2 行（定义输入端口最低比特对应第 0 行）。此时可根据行值确定按键所在行的行号，具体过程为：行线数据循环右移一位，此时若进位位为 0 则表明第 0 行线的 0 号键闭合；否则继续循环右移，直至找出闭合键为止。

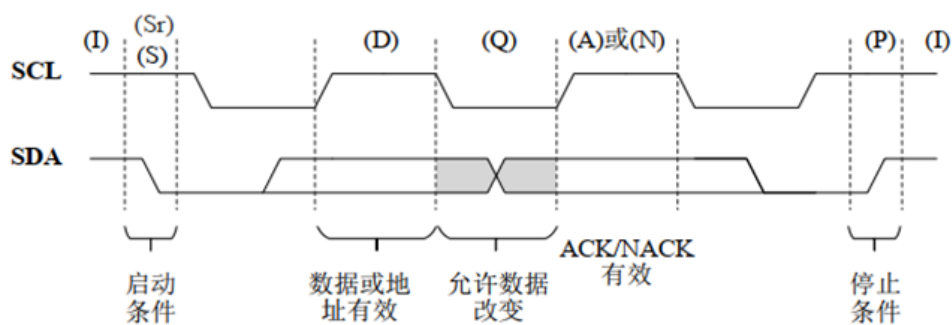
4.53 简述 LED 数码管的动态显示原理。

动态显示方式也称为扫描显示方式，可以使用较少的信号线实现显示控制，动态显示的基本思路是让各个数码管轮流显示，每位数码管的点亮时间约 $1\sim 2\text{ms}$ ，由于视觉暂留现象及发光二极管的余辉效应，尽管各个数码管并非同时点亮，但给人的感觉是所有数码管同时显示。在动态显示方式下，各个数码管的对应段的输入控制端并连在一起，因此无论数码管的个数是多少，需要的口线数目都是 8，该端口称为段选口。各个数码管的公共端各自连接一根口线，该口线称为位选口。当数码管的个数为 N 时，则需要 N 根位选口口线，因此动态显示方式总共需要的口线数量为 $8+N$ 。

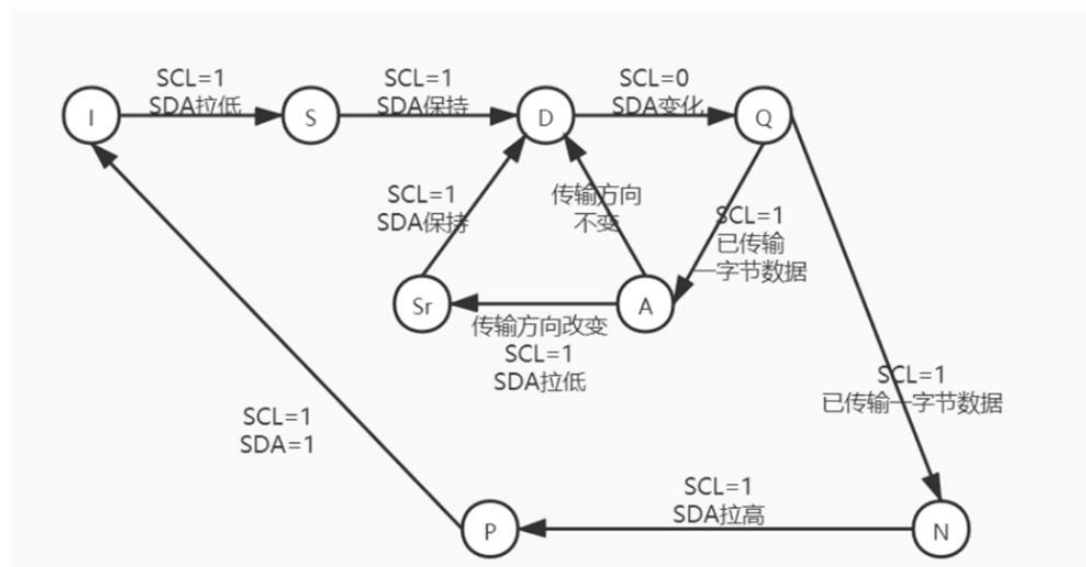
4.58 异步串行通信系统中，采样数据时为什么要在数据位的中间？

在上升沿和下降沿采样可能会采样到相邻位的信号，在数据位中间采样可以提高采样准确率。

4.61 描述 I²C 总线协议中的状态，并画出状态转移图。



总线空闲I、启动数据传输S、停止数据传输P、重新启动Sr、数据有效D、等待/数据无效Q、应答A、不应答N



5.1 计算机中的“ISA”和“ μ arch”各是什么意思？两者之间有何联系？

ISA：指令集体系结构，用来描述软件如何使用硬件的一种规范和约定

μ arch：微架构，对应于 ISA 的硬件实现方式，。

相同的 ISA 的处理器可以有不同微架构，ISA 是介于微架构硬件实现和软件的桥梁。

5.2 请简述哈佛结构的主要优缺点。

优点：取指和数据存取可以通过两套独立的总线同时进行，从而减小指令流水线发生资源冲突的概率。

缺点：哈佛结构较为复杂，与外设以及扩展存储器的连接难度较大。

5.3 TCM 与高速缓存 Cache 有什么区别？

TCM 具有物理地址，需要占用内存空间，无 cache 的不可预测性

5.5 ARM 指令集、Thumb 指令集和 Thumb-2 指令集之间的主要区别是什么？

ARM 指令集是 32 位，thumb 指令集只有 16 位，虽然 thumb 指令集在指令功能方面不如 ARM 全面，但是提高了代码密度，降低系统成本；另外大多数外设的接口都是 8 位或者 16 位的，利用 thumb 指令集可以提高传输效率。Thumb-2 指令集结合了 thumb 和 ARM 指令集的优点。

5.6 MMU 和 MPU 的功能有何异同？

MMU：内存分页管理+分区域访问权限管理+虚拟地址 VA 到物理地址 PA 的转换，适用于多用户系统。

MPU:内存分区域访问权限管理，适用于要求对处理时间有明确要求的实时系统。

Memory Management Unit 和 Memory Protection Unit，均是用于内存管理，且基本功能大体上一致，但 MMU 较 MPU 更为先进，具有 MPU 所不具备的内存分页转化和虚拟地址到物理地址的转换，覆盖 MPU 的功能且开销更低，适用于多系统。

5.9 Cortex-M系列处理器定义的存储器映射关系是固定不变的，这样做有何利弊？

利：相应的电路功能简单一些，相关电路可以简化；提高代码的可重用性和软件可移植性。

弊：相应的存储扩展只能在对应的区域进行，缺乏灵活性。

5.10 Cortex-M3与Cortex-M4使用两个堆栈的目的是什么？在中断响应时，程序断点和程序状态寄存器的内容保存在哪个堆栈中？

使用两个堆栈是为了服务于不同的操作模式和特权访问等级，处理模式总是使用MSP，线程模式可以使用MSP或PSP。程序断点和程序状态寄存器的内容保存在MSP中

5.10 答：Cortex-M3/M4 处理器有 2 种操作模式：处理模式和线程模式，分别对应着不同
不同特权等级等以及不同操作。
使用 2 个堆栈，主堆栈和线程进程堆栈，在基址
其目的是：在线程模式可以切换使用独立的进程栈指针，使应用任务的栈
空间和操作系统的主栈空间独立，提高系统健壮性和可靠性。

当处理器进入中断响应时，进入处理模式，使用主栈指针。程序断点和状态寄存器的内容
保存在主堆栈中。

5.11 Cortex-M3/M4的CODE区选用总线互连矩阵与总线复用器有什么区别？

总线矩阵：I-CODE对FLASH的取指操作与D-CODE对SRAM的数据存取操作可以同时进行

总线复用器：I-CODE和D-CODE对CODE区域的访问只能分时进行，数据传送不具有并行性。

5.13 Cortex-M3/M4从SRAM域读取指令执行时有什么缺点？

从SRAM区域读取指令和从CODE区域利用I-CODE读取指令相比，效率较低

5.14 I-Code和D-Code总线全部连接到同一片Flash芯片上会有什么问题？

会产生冲突，无法做到并行操作，降低效率。

5.15 私有外设总线（Private Peripheral Bus, PPB）基于哪种总线协议，有何特点？

APB总线协议，访问区域为外部私有外设区域，PPB总线一般专门连接调试组件，不用于普通外设。

5.16 如果非特权线程试图访问内核私有区域，将会导致哪一类异常？如果Cortex-M3使用了一条SIMD运算指令，结果又将如何？

编号4 MemManage错误

由于cortex-M3没有浮点运算，DSP等协处理器，所以会触发用法错误（编号6）

5.17 在Cortex-M3/M4中，寄存器R0~R12有何异同？如果这些寄存器都是空闲的，你觉得首先使用哪些？为什么？

R0~R7 低位寄存器(许多16位的thumb指令只能访问低位寄存器) R8~R12高位寄存器(可用于32位指令和少数几个16位指令) 相同点：都是通用寄存器。

为了能够实现汇编程序与C语言程序的相互调用，ARM公司制定了AAPCS（ARM Architecture Procedure Call Standard）规范：R0~R3用于子程序之间的参数传递；R4~R11用于保存子程序的局部变量；R12作为子程序调用中间寄存器。

R0~R7 是 8 个低位寄存器，R8~R12 是 5 个高位寄存器，R0~R3 用于子程序之间的参数传递，R4~R11 用于保存子程序的局部变量，R12 作为子程序调用中间寄存器。优先低位寄存器，低位使用较频繁，优先低位可以降低功耗。



5.19 某段程序需要跳转到0x0100 0000执行，有人写了如下两行汇编指令代码：

```
MOV R0, #0x0100 0000
```

```
MOV R15, R0
```

请问这样会有什么问题？

无论使用跳转指令还是直接写PC寄存器，写入值必须是奇数，确保其最低位是“1”，以表示其处于Thumb状态，否则将被认为试图转入ARM模式，从而导致出现错误异常

5.20 请说明特殊寄存器PRIMASK和FAULTMASK寄存器的异同。

异：与PRIMASK不同的是，FAULTMASK无需主动清理，当错误处理程序运行结束返回时，会自动复位FAULTMASK。PRIMASK，当最低位被置位（写入1）后，将屏蔽除复位、NMI和硬件错误以外所有的（优先级数值大于0的）系统异常和外部中断同：FAULTMASK硬件错误异常也被屏蔽。

同：都属于实现1位基于优先权异常/中断寄存器。

5.22 某基于Cortex-M4的SOC芯片共有64级外部中断，BASEPRI寄存器的宽度共有几位？如果想屏蔽所有优先级大于16的中断，请写出对BASEPRI寄存器进行设置的汇编指令。如果想屏蔽所有优先级大于0的中断，又该如何设置？

BASEPRI:7:2共6位

MOV R0, #0x 0000 0044 或者MOV R0, #0b 010001

MOV BASEPRI, R0

MOV R0, #0x 0000 0001

MOV PRIMASK, R0

5.23 有人写了一段对Cortex-M4的进程栈进行初始化的代码，其中PSP的初始值设为0x8765 4321，并且使用了如下一条语句：“MOV PSP, R0”对PSP进行赋值（其中R0=0x8765 4321）。这样做存在哪些问题？请逐一说明。

由于堆栈操作是以字为单位的，所以堆栈要字对齐，而PSP的初始值并没有做到字对齐。

MSP, PSP只能用特殊寄存器指令MRS, MSR指令访问。

5.25 在特权线程模式下如何切换到非特权线程模式？在非特权线程模式下能否采用类似方法切换到特权线程模式？为什么？

在特权线程模式下可以直接修改CONTROL寄存器nPRIV=1，就可以直接进入非特权线程模式。

在非特权线程模式下，首先通过异常状态进入异常处理，在异常处理阶段修改CONTROL寄存器nPRIV=0，然后就进入特权线程模式。

5.29 Cortex-M3存储空间的哪些区域支持位段（bit-band）操作？

SRAM区域，片上外设区域支持bit-band操作



5.31 写出利用位段操作读取0x4000 1000的第3位的代码。

```
LDR R0, =0x4202 0008
```

```
LDR R1, [R0]
```

5.32 存储器访问属性包括哪些？

可缓冲：当处理器继续执行下一条指令时，对存储器的写操作可以由写缓冲执行

可缓存：读存储器所得到的数据可被复制到缓存，下次再访问时可以从缓存中取出这个数值从而加快程序执行

可执行：所得到的数据可被复制到缓存，下次再访问时可以从缓存中取出这个数值从而加快程序执行

可共享：这种存储器区域的数据可被多个总线主设备共用。存储器系统需要在不同总线主设备之间确保可共享存储器区域数据的一致性。

5.35 Cortex-M系列处理器不会改变代码的执行顺序，因而不需要存储器屏障指令，这个观点对吗？为什么？

不正确。由于cortex-M系列存储器引入缓存，虽然存储器系统不会改变指令执行顺序，但是顺序执行的指令的存储器访问操作完成时间先后顺序不定，因此需要存储器屏障指令来保证代码的执行顺序。



5.36 处理器进入异常处理子程序之前保护现场需要把哪些寄存器的值保护起来？

PSR, PC, LR, R0~R3, R12

5.38 解释Cortex-M处理器的中断优先级分组机制。

8位的优先级配置寄存器分为两部分：分组优先级和（组内）子优先级。

由于分组优先级和子优先级可以配置不同宽度的组合，有效提高中断优先级配置的灵活性。

分组优先级对应抢占优先级，在相同的分组优先级下，具有更高子优先级的异常会被优先处理

5.39 解释向量表重定位机制。

向量表重定位使用VTOR指示向量表的位置，保存向量表相对于存储器的偏移量
处理器通过修改VTOR值修改向量表的起始位置，从而实现向量表重定位。

第 6 章作业

6.6 一条机器指令应该包含哪些要素？

机器指令是用二进制位串来表示的, 表示一条特定指令的二进制位串(比特串)称为指令字, 简称指令。指令字需要包含操作码、操作数(可能有多个)和操作数地址等字段。

6.12 指令中的操作数可以存放在哪些地方？

表 6.8 寻址方式的分类

操作数位置	可能的寻址方式	英文原称
内含在指令中	1) 立即数寻址	Immediate addressing
存放在寄存器中	2) 寄存器寻址	Register addressing
	3) 寄存器移位寻址	Shifted register addressing
存放在存储器数据区	4) 寄存器间接寻址	Register addressing
	5) 寄存器偏移寻址	Offset addressing
	6) 前变址寻址	Pre-indexed addressing
	7) 后变址寻址	Post-indexed addressing
	8) 多寄存器寻址	multiple registers
	9) 堆栈寻址	SP addressing
存放在存储器代码区	10) PC 相对寻址	PC-related addressing (Literal)

6.15 对比寄存器偏移寻址、前变址寻址和后变址寻址的异同。

相同点:

操作数地址由一个寄存器中存放的数值与指令中给出的地址偏移量相加得到

不同点:

寄存器偏移寻址: 不用更新基址寄存器

前变址寻址: 执行指令后自动把基址与偏移相加形成的操作数地址写回到基址寄存器中。基址寄存器中的数值和立即数偏移量的加和计算发生在寻址前。

后变址寻址: 在执行指令的时候, 操作数地址从基址寄存器获取, 指令执行后再将操作数地址加上偏移量生成一个新的地址, 并将该新地址写入基址寄存器。指令中的偏移量在存储器访问期间不会用到偏移, 而是在数据传输结束后更新基址寄存器。

6.20 解释汇编语法“LDM {addr_mode} <Rn>{!}, <registers>”中各部分要素的含义。

LDM: 使用多寄存器寻址方式。

{addr_mode}: 可选后缀:

IA: 每取一个操作数后基址寄存器递增

IB: 每取一个操作数前基址寄存器递增

DB: 每取一个操作数前基址寄存器递减

DA: 每取一个操作数后基址寄存器递减

<Rn>: 基址寄存器

{!}: 可选项(!)表示需要将修改后的地址写入基址寄存器<Rn>

<registers>: 需要载入数据的寄存器集合

6.24 解释堆栈寻址和 PC 相对寻址两种方式中基地址各存放在哪里？

堆栈寻址：堆栈指针寄存器 SP

PC 相对寻址：程序计数器寄存器 PC

6.34 什么是符号扩展？

⁹ **符号位扩展**：低位宽的数存到更高位宽的存储单元时，需要考虑多出来的位填充什么信息。例如，8 位的有符号数 1111 0000B 存储到 16 位的寄存器中，寄存器的低 8 位存放 1111 0000，高 8 位填充 1111 1111，这种操作称作符号位扩展，常简称符号扩展。有符号数为负数时高 8 位填充的都是“1”，有符号数为正数时，则符号位扩展会将高 8 位均填为“0”。与之对应还有一个术语，**零扩展**。一个 8 位的数 b 零扩展至 16 位就是将高 8 位全部填充“0”。习惯上把符号位扩展和零扩展统称为数据扩展。

6.35 MOV 指令是否可以完成从一个存储器单元到一个寄存器的数据传送？为什么？

不行，因为 MOV 指令用于将数据从一个寄存器送到另外一个寄存器。存储器访问指令为 LDR 和 STR。

6.40 为什么“BL”或“BLX”指令适用于函数调用，而“B”指令不适合。

BL 和 BLX 都能将返回地址保存到 LR (R14) 中，而 B 没有给 LR 赋值，即跳转之后的 LR 还是跳转之前的 LR 值。

6.42 “CBZ”指令的作用与“CMP”指令组合“BEQ”指令有什么区别？

CBZ 合并了和零比较以及条件跳转操作，相当于 CMP 指令和 BEQ 指令的功能组合，区别在于 APSR 的值不受 CBZ 的影响。

另外，CBZ 指令只支持前向跳转，不支持向后跳转，跳转范围为当前指令后的 4~130 字节；CBZ 只能使用 R0~R7。

6.43 解释饱和和溢出的区别。

饱和：把超出动态范围的数据强制置为最大允许值。

溢出：在数据超出允许范围时将数据的最高位去掉。

溢出操作可能会使波形出现严重的畸变。饱和运算可以减小畸变，但畸变仍然存在，相比于溢出操作至少保留了信号波形的大致形状。

6.45 为什么 ARM 的存储器访问指令要提供非特权加载和存储功能？

8. 非特权等级加载和存储

ARM 处理器区分特权和非特权访问等级。运行于非特权访问等级的程序无法访问特权访问等级程序中的数据。例如在有操作系统的情形下，操作系统的代码运行在特权访问等级，而普通用户的代码则运行在非特权访问等级。用户程序通过操作系统提供的 API 函数访问存储器的时候，可能会因目标存储位置的访问等级设置而无法访问。故 ARM 中提供了一组特殊的加载和存储指令，在特权访问等级程序中使用这些指令加载或保存的数据，其权限等同于非特权访问等级的程序。

第 7 章作业

7.3 编写一个完整 ARM 汇编程序实现如下功能：当 $R3 > R2$ 时，将 $R2+10$ 存入 $R3$ ，否则将 $R2+100$ 存入 $R3$ 。

写法不唯一，注意程序完整性和格式对齐

```
AREA BUF, DATA, READWRITE
Num1 DCD 0x01
Num2 DCD 0x02
AREA RESET, CODE, READONLY
ENTRY
START LDR R0, =Num1 ;这里 START 不是关键字，只是标号，可以随便起，也可以不写
      LDR R2, [R0]
      LDR R1, =Num2;
      LDR R3, [R1];
      CMP R3, R2
      ADDHI R3, R2, #10
      ADDLS R3, R2, #100
      END
```

7.9 编写完整 C 程序并利用汇编子程序计算 $N!$ ($N \leq 10$)。

参考例 7.48，调用汇编函数，计算 $N!$

```
//main.c
extern void NMUL (int n); //需要调用的汇编函数原型并加 extern 关键字
int main (){
    int n=10;
    int result;
    result = NUML(n);
    return result ;
}
```

汇编子程序，计算 $N!$

```
;汇编语言源程序 Nmul.s，汇编文件和*.c 文件在同一工程中
AREA Nmul, CODE, READONLY
EXPORT NMUL
NMUL ;必须与 EXPORT 后面标号一致
    MOV R1, R0 ;将 R0 中的数值（即 N 的值）传到 R1
    MOV R0, #1 ;R0 置为 1 以存放累乘结果
    CMP R1, #0 ;考虑  $N=0$  的情况。CMP 和 BEQ 用法参考例 7.41
    BEQ NEQZERO ;若 R1 中的值为 0，跳转到 NEQZERO 处执行
LOOP MUL R0, R0, R1 ;若 R1 中的值不为 0，则计算累乘
    SUBS R1, R1, #1 ;计数器-1
    CMP R1, #0
```

```

        BNE  LOOP          ;BNE 用法参考例 7.44
        B   STOP           ;完成计算后跳转到 STOP
NEQZERO MOV  R0, #1
STOP  MOV  PC, LR
END

```

7.10 编写完整汇编程序调用 C 函数计算 N! ($N \leq 10$)。

计算 N! 的 C 函数

```

int NMUL(int n){
    int result = 1;
    if (n==0) return result;    //考虑 N= 0 的情况
    for (int i=1; i<=n; i++){
        result=result*i;
    }
    return result;
}

```

汇编程序参考例 7.49，其中调用了 C 函数 NMUL 用于计算 N!

```

AREA  RESET, CODE, READONLY
ENTRY
IMPORT  NMUL
LDR  SP, =0x20000460      ;设置堆栈指针
MOV  R0, #10              ;设定 N
BL  NMUL
LDR  R4, =0x20000000
STR  R0, [R4]
END

```

第 8 章作业

第 8 章作业说明

- ☐ 以下以灰色双删除线标识的不做。
- ☐ *标识的为选作题（非教材上作业题，部分题涉及查阅芯片手册），选做题要求是 6 题中至少选 2 题做。

-----PART1-----

8.3 ~~嵌入式开发与通用计算机系统相比,有何独特之处?交叉开发环境的主要组成部分和特点是什么?~~

嵌入式系统的开发环境称为交叉开发环境(cross development environment)，主要由宿主机 host、目标机 target 及它们之间的连接构成。

综上所述，嵌入式系统的开发环境中，宿主机（通用 PC）和目标机（嵌入式系统）是不同的机器。嵌入式软件在宿主机上使用嵌入式开发工具进行编写、编译、链接和定位，生成可在目标机上执行的二进制代码，然后通过 JTAG 接口、串口或网口将代码下载到目标机上调试，调试完成后，将最终调试好的二进制代码烧写到目标机微处理器的 ROM 中运行。

8.7 ~~何为引脚功能复用?有何意义?如何实现,请以 STM32F103 为例,举例具体说明。~~

ARM 处理器的片内资源丰富功能强大，器件功能一般需通过引脚表现出来。如果将芯片全部功能都对应安排专用引脚，则引脚总数会相当庞大，既增加了产品成本还加大了应用成本，而且在某个具体的实际应用中基本不会用到器件全部资源。为此，特意将某几个功能分配到同一个引脚，通过编程分别将芯片不同功能引出（使之在某个时刻对应某个功能），则大大减少了引脚总数，这种方法称为引脚功能复用技术。

意义：大大减少了引脚数。如果将芯片全部功能都安排专用引脚，庞大引脚数量会提升产品成本，装焊困难，加大应用成本，而且实际应用中极少会用到器件全部资源。

8.11 请设计一个 STM32 最小系统。
给出 STM32 最小系统必要的组成部分即可，不需要画出下图完整的电路图

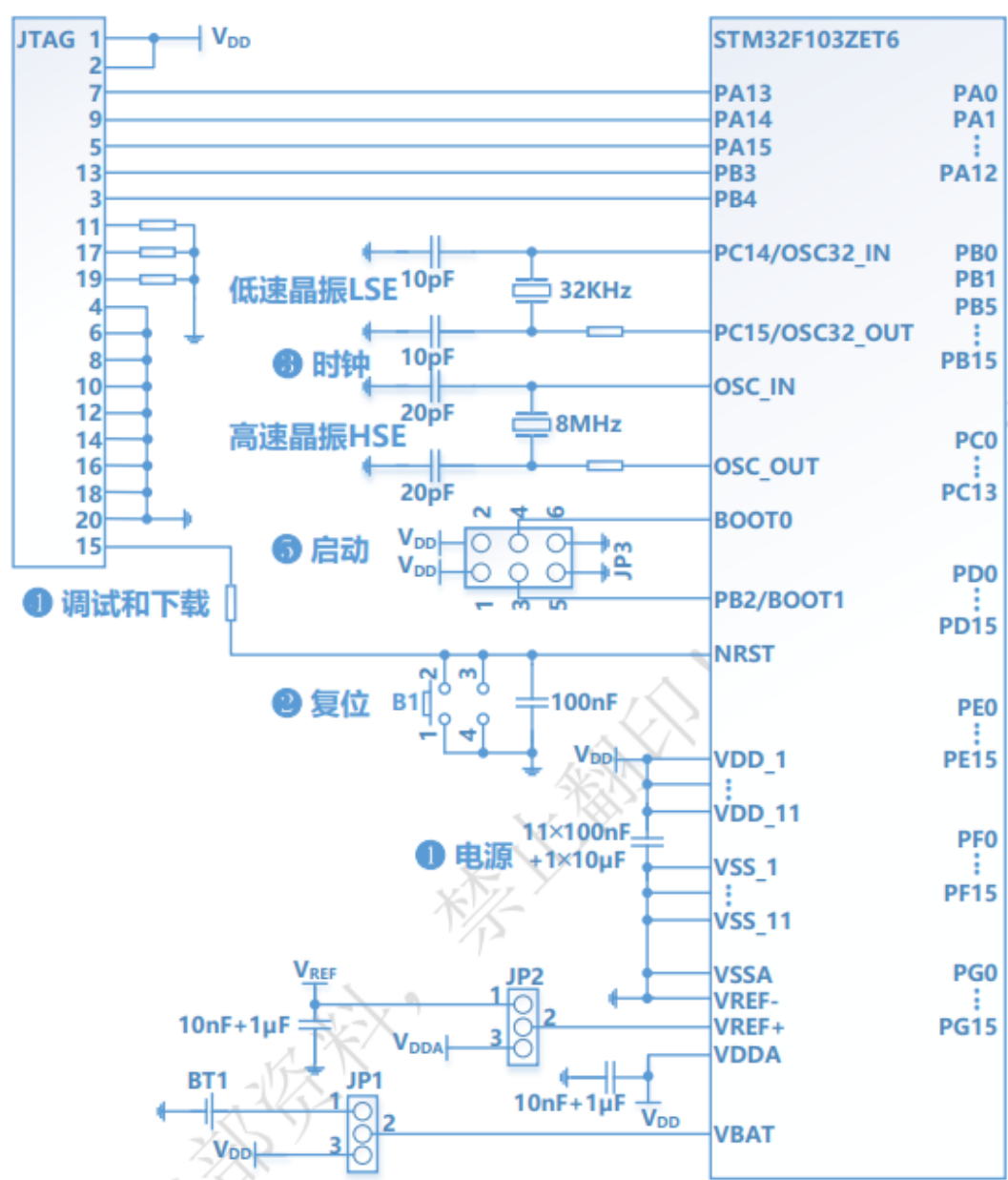


图 8.20 STM32F103 最小硬件系统原理图

8.12 STM32F103 的复位电路有何功能？常见的复位方式有哪些？

2. 复位电路

当微处理器上电时，电压不是直接跳变到微处理器可工作的范围(如 3.3V)，而是一个逐步上升的过程。此时，微处理器无法正常工作，会引起芯片内程序无序执行。另外，当微处理器的供电电压波动较大时也可能会发生同样的情况。因此这时需要复位电路给它延时，使微处理器保持复位，暂不进入工作状态，这样可防止 CPU 执行错误指令，确保 CPU 及各部件处于确定的初始状态，直至电压稳定。显而易见，复位电路会直接影响到系统稳定性和可靠性。未添加复位电路或复位电路设计不可靠可能会引起“死机”及“程序跑飞”等现象。

STM32F103 支持 3 种复位形式，分别为系统复位、电源复位和备份区域复位。

8.13 STM32 的启动模式有几种？BOOT 引脚有几个？分别如何设置？

表 8.19

BOOT0	BOOT1	启动模式	STM32F103 存储空间起始地址 0x00000000 的映射
0	x	从用户 Flash 启动	映射到用户 Flash(主 Flash)的起始地址
1	0	从系统 Flash 启动	映射到系统 Flash 的起始地址
0	1	从片内 SRAM 启动	映射到片内 SRAM 的起始地址

8.14 STM32F103 的时钟有哪几种产生方式？各有什么特点？实际应用中一般如何做以实现系统的时钟驱动。

STM32F103 内置内部 RC 振荡器，为内部锁相环 PLL 提供时钟，使 STM32F103 可在 72MHz 的满速状态下运行。但内部 RC 振荡器相比外部晶振，不够准确稳定，应尽量使用外部主时钟源。

外部主时钟源主要作为内核和外设的驱动时钟，一般称为高速外部时钟信号 HSE(High Speed External clock signal)。HSE 可由以下两种时钟源产生。

- ❑ 外部晶体/陶瓷谐振器。时钟电路由外部晶振和负载电容组成。负载电容的值需根据选定晶振进行调节。电容位置应尽可能地靠近晶振引脚，以减小输出失真和启动稳定时间。外部晶振频率可以是 4~16MHz（常选用 8MHz）。此模式能产生非常精确而稳定的主时钟，是主时钟源的首选。HSE 晶振可通过设置时钟控制寄存器 RCC_CR.HSEON 位来启动和关闭；RCC_CR.HSERDY 位用来指示高速外部振荡器是否稳定，在启动时，直到此位被硬件置 1，时钟才被释放出来；若时钟中断寄存器 RCC_CIR 中允许中断，将会产生相应的中断。
- ❑ 用户提供的外部时钟信号。此外时钟源频率最高可达 25MHz（可以是占空比为 50% 的方波、正弦波或三角波）。时钟信号须连到 OSC_IN 引脚，同时保证 OSC_OUT 引脚悬空。可通过设置 RCC_CR.HSEBYP 位和 RCC_CR.HSEON 位来选择这一模式（即 HSE 旁路）。

8.16 参照 STM32 的时钟树，请做以下思考，并解释 1) 设计成这种形式的主要目的和理由为何？ 2) 从左至右，相关时钟依次可分为大致哪几种？ 3) 时钟输出的使能有何意义，并以一个实例（如 TIM4）说明一下大体流程。

(1)

微处理器运行须依赖周期性的时钟脉冲，往往由一个外部晶体振荡器提供时钟输入为始，最终转换为多个外设的周期性运作为末，这种时钟“能量”扩散流动的路径，犹如大树的养分通过主干流向各个分支，因此常称之为“时钟树”。

(2)

由图 8.22 可见，从左至右，相关时钟依次可分为 3 种：输入时钟、系统时钟和由系统时钟分频得到的其他时钟（主要是各种外设时钟），现分别介绍如下。

(3)

时钟输出的使能的意义：让用户可以精确地控制时钟，需要使用时使能对应时钟，不需要使用时关闭，以减小能耗。

8.21 以 STM32 为例，描述程序开发的一些主要模式，各自有什么特点，优势和缺点都体现在哪些地方？~~实际应用中该如何取舍。~~

8.4.3 嵌入式软件开发模式

以 STM32F103 微处理器为例，简介常用的嵌入式软件开发模式。大体有如下几种：（1）基于寄存器开发；（2）基于（ST 官方提供）固件库开发；（3）基于嵌入式操作系统开发；（4）中间件开发。

具体优缺点根据教材内容总结即可。

8.23 以 STM32 固件库为基础利用工程方式，进行嵌入式软件应用开发，有哪些主要步骤？请略述之。

2. 以工程方式嵌入式软件开发流程

工程方式的开发流程一般可分为以下几步：

- ①从 ST 官网下载相应标准外设库（即固件库）。
- ②下载和安装支持标准库的嵌入式开发工具（如 KEIL MDK 等）。
- ③以官方工程模板为基础，根据所用微处理器型号和软件编译设置，更改相关配置选项。
- ④编写程序代码（主要是工程目录下 User 组中的两个：main.c 和 stm32f10x_it.c）。
- ⑤编译和链接工程。
- ⑥使用软件模拟仿真或下载硬件运行（仿真器跟踪运行）等方式调试程序。
- ⑦将最终生成的可执行程序下载至目标微处理器内置 ROM 中，复位，运行。

8.59 *简述 STM32F103 的启动过程。

启动过程是指微处理器从上电复位到开始执行用户应用程序(对于 C 应用程序来说，就是跳转向 main 函数)之间的那段时间，期间所完成的主要工作总结如下。

1) 根据 BOOT0 和 BOOT1 引脚选择启动存储器映射

上电后，首先根据 BOOT0 和 BOOT1 引脚的电平高低决定从哪个存储区启动，即选择将哪个存储区映射到 0 地址区（参见前述表 8.19 和图 8.24）。通常，选择从片内用户 Flash 启动，即将用户 Flash 的起始地址 0x08000000 映射到 0 地址。

2) 从地址 0x00000000 处取出栈顶指针值放入 MSP

假设上一步中设置为从片内用户 Flash 启动（最为常见），那么从地址 0x00000000（实际上是地址 0x08000000，如图 8.25 所示）处取一个字(4B)，作为主堆栈的栈顶指针送入 MSP。

在执行复位异常处理程序前，初始化 MSP 是必要的。因为可能第 1 条指令还没来得及执行，就发生了 NMI 或是其他 fault。MSP 初始化好后即为异常服务程序准备好了堆栈。

3) 从地址 0x00000004 处取出复位异常服务程序的入口地址放入 PC

从地址 0x00000004 处取出一个字作为 PC 的初始值，这个值即是复位向量。类似地，如果在第一步中设置为从片内用户 Flash 启动，那么实际上微处理器将从地址 0x08000004 处取出一个字送入 PC。此时，存储空间和寄存器如图 8.25 所示。由于 STM32F103 是在 Thumb 态下执行，所以向量表中的每个地址都必须把 LSB 置 1。因此，图 8.25 中复位向量使用 0x08000145 来表示复位异常处理程序的地址 0x08000144，即 0x00000144。以上信息可从工程编译、链接生成的.map 文件看到。

4) 执行复位异常服务程序

复位异常服务程序 Reset_Handler 在启动代码中定义，负责初始化时钟系统和 C 应用程序运行环境，并跳转到应用程序的 main 函数，其具体执行过程如图 8.26 所示。

8.25 ST 固件库中，函数的命名规则有哪些特点？

题里只问了函数的命名规则，只需要回答下面的第（5）点即可

ST 固件库的命名规则

(1) PPP 表示任一外设缩写，PPP 可是 GPIO、TIM、NVIC、EXTI、USART、SPI 和 I²C、DMA、ADC 等。

(2) 源程序文件和头文件命名都以“stm32f10x_”作为开头，例如：stm32f4xx_exti.h。

(3) 常量仅被应用于一个文件的，定义于该文件中；被应用于多个文件的，在对应头文件中定义。所有常量都由英文字母大写书写。例如 GPIO_Mode_OUT 被定义在 stm32f4xx_gpio.h 中，其值为 0x01。

(4) 寄存器作为常量处理。他们的命名都由英文字母大写书写。在大多数情况下，他们采用与缩写规范一致。例如 core_cm4.h 中定义了 NVIC 的相关寄存器。

(5) 外设函数的命名以该外设的缩写加下划线为开头。每个单词的第一个字母都由英文字母大写书写，例如：EXTI_Init。在函数名中，只允许存在一个下划线，用以分隔外设缩写和函数名的其它部分。具体例子参考讲义 8.8.6 节的小节中的 STM32F10x 标准外设库函数的分类和命名。

参考资料：

STM32 固件库的下载（这篇资料后面给出了固件库的移植方法，即创建实验工程中的 CORE,FWLIB,SYSREM,USER 文件夹）

<https://blog.csdn.net/xzzszka/article/details/123768623>

STM32 固件库详解（这篇资料里给出了固件库的内容和命名规则）

https://blog.csdn.net/weixin_30477293/article/details/95117663

8.26 STM32 F103 的 GPIO 有哪 8 种工作模式？~~略述每种模式的特点。~~

4 种输出模式：

普通推挽输出 PP，普通开漏输出 OD，复用推挽输出 AF_PP，复用开漏输出 AF_OD

4 种输入模式

上拉输入 IPU，下拉输入 IPD，浮空输入 IN_FLOATING，模拟输入 AIN

8.27 GPIO 的复用功能重映射有何意义？如何实现的，举一个例子说明。

通过利用复用功能的 I/O 重映射，至少得到以下几点好处：可分时复用某些外设，虚拟增加了端口数量；可优化引脚配置和 PCB 布线设计；减少信号交叉干扰。

为实现复用功能重映射，需进行以下操作，如图 8.31 所示。

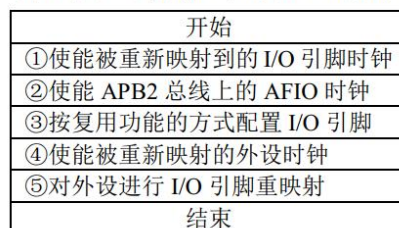


图 8.31 GPIO 的复用功能重映射操作流程

例如，要将 USART2 的 Tx 和 Rx 从默认的引脚 PA2 和 PA3 重新映射到引脚 PD5 和 PD6，根据图述步骤，可使用库函数编程如下：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIO,ENABLE); //使能引脚 PD5 和 PD6 的时钟

RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO,ENABLE); //使能引脚 PD5 和 PD6 上 AFIO 时钟
//根据 USART2 的 Tx 和 Rx，分别将引脚 PD5 和 PD6 设置为推挽复用和浮空输入模式
GPIO_InitStructure.GPIO_Pin=GPIO_Pin_5;
GPIO_InitStructure.GPIO_Mode=GPIO_Mode_AF_PP;
GPIO_Init(GPIO, &GPIO_InitStructure);
GPIO_InitStructure.GPIO_Pin=GPIO_Pin_6;
GPIO_InitStructure.GPIO_Mode=GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIO, &GPIO_InitStructure);
//使能要进行 I/O 引脚重映射的外设 USART2 的时钟
RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART2,ENABLE);
GPIO_PinRemapConfig(GPIO_Remap_USART2,ENABLE); //进行 USART2 的 I/O 引脚重映射
以上语句中所用库函数可参考本章后续相关小节及技术手册。
```

8.28 如果使用 GPIOC6 接口为推挽方式输出，最高频率 50MHz，分别使用寄存器模式和库函数模式，如何进行编程。

8.5.3 节

寄存器模式

GPIOC6 接口为推挽方式输出，最高频率 50MHz

GPIOC_CRL 的 CNF6[1:0] 为 00，和 MODE6[1:0] 为 11

GPIOC_CRL&=0xF0FFFFFF; //清除对位 6 的配置

GPIOC_CRL|=0x03000000; //写位 6 的配置为 0011b（即 3）

库函数模式：

```
GPIO_InitTypeDef GPIO_InitStructure;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC,ENABLE);
GPIO_InitStructure.GPIO_Pin=GPIO_Pin_6;
GPIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP;
GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz;
GPIO_Init(GPIOC, &GPIO_InitStructure);
```

实验讲义内的写法：

```
GPIO_InitStructure.GPIO_Mode=GPIO_Mode_OUT; //普通输出模式
GPIO_InitStructure.GPIO_OType=GPIO_OType_PP; //推挽输出
```

-----PART2-----

8.60 *假设 TIMxCLK 为 72MHz，有如下代码。请问：(1) 启动 TIM2 多长时间后 TIM2_CNT.CNT 数值为 249？(2) 启动 TIM2 多长时间后 TIM2_ARR.ARR 与 TIM2_CNT.CNT 数值相同？

```
#include stm32f10x_tim.h

void main()
{
    //.....
    TIM_TimeBaseInitTypeDef my_timer;
    my_timer.TIM_Prescaler= 71999;
    my_timer.TIM_Period= 999;
    my_timer.TIM_CounterMode= TIM_CounterMode_Up;
    TIM_TimeBaseInit (TIM2, & my_timer);
    //.....
}

typedef struct
{
    uint16_t TIM_Prescaler;           //定时器预分频：0~0xFFFF
    uint16_t TIM_CounterMode;         //选择计数器模式：Up/Down 向上/向下计数；
                                     //CenterAligned1/2/3 中央对齐模式 1/2/3
    uint16_t TIM_Period;              //在下一个更新事件装入 ARR 值：0~0xFFFF
    uint16_t TIM_ClockDivision;       //CK_INT 与死区发生器以及数字滤波器采样频率分频系数：
                                     //TIM_CKD_DIV1/2/4
    uint8_t TIM_RepetitionCounter;    //重复定时器值（计数周期数）
} TIM_TimeBaseInitTypeDef;
```

PSC=71999，ARR=999，Tclk=72MHz 计数模式为向上计数

TIM2_CNT.CNT 数值为 249 经过的时间为 $249 * (71999 + 1) / 72\text{MHz} = 0.249\text{s}$

TIM2_ARR.ARR 与 TIM2_CNT.CNT 数值相同时即 CNT=999，经过时间为 $999 * (71999 + 1) / 72\text{MHz} = 0.999\text{s}$

注意：在向上计数过程中，并不是 CNT 计数到等于 ARR 的瞬间就清零，而是在下一个计时器周期发生溢出并产生更新事件后才清零。可参考下图

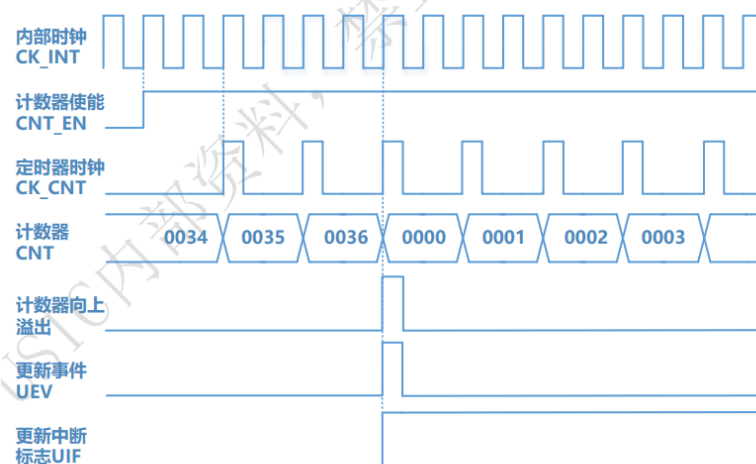


图 8.41 向上计数模式下通用定时器的计数时序图

8) 定时计算公式

$$T_{out}(\mu s) = ((ARR+1)(PSC+1))/T_{clk}(MHz)$$

例如，计时 1S，可由下算出 ARR 的值：假设，输入时钟频率 72MHz，PSC 预分频系数 35999，
 $((ARR+1)(PSC+1))/T_{clk}(MHz) = (ARR+1) * (35999+1)/72M = 1(s)$ ；得出 ARR=1999

8.31 简述通用定时器的输入捕获过程。

8.6.3 STM32F103 通用定时器 TIM2~TIM5

输入捕获模式可用来测量（输入信号）脉冲宽度或频率。简单说就是通过检测 CHx 上的边沿信号，在边沿信号发生跳变（比如上升沿/下降沿）的时候，（利用中断功能）将当前定时器的值（CNT）锁存到对应通道的捕获/比较寄存器 CCRx 中，把前后两次捕获到的 CCR 的值相减，就可以计出脉宽或频率。除了 TIM6 和 TIM7，其他 6 个定时器都有输入捕获功能。

8.32 参照书中例子，采用 TIM2 通道 2 进行频率测量，利用库函数实现其设置。

//TIM2 的 2 通道设置：

```
TIM_ICInitTypeDef TIM_ICInitStructure;
```

```
TIM_ICInitStructure.TIM_ICMode=TIM_ICMode_ICAP; //配置为输入捕获模式
```

```
TIM_ICInitStructure.TIM_Channel=TIM_Channel_2; //选择通道 2
```

```
TIM_ICInitStructure.TIM_ICPolarity=TIM_ICPolarity_Rising; //输入上升沿捕获
```

```
TIM_ICInitStructure.TIM_ICSelection=TIM_ICSelection_DirectTI; //通道方向选择
```

```
TIM_ICInitStructure.TIM_ICPrescaler=TIM_ICPSC_DIV1; //每次检测到输入捕获就触发一次捕获
```

```
TIM_ICInitStructure.TIM_ICFilter=0x0; //不分频
```

```
TIM_ICInit(TIM2,&TIM_ICInitStructure);
```

//输入捕获配置：

```
TIM_SelectInputTrigger(TIM2,TIM_TS_TI1FP1); //选择滤波后 TI1 作为输入触发源，触发下面程序的复位
```

```
TIM_SelectSlaveMode(TIM2,TIM_SlaveMode_Reset);
```

//复位模式-选中的触发输入（TRGI）的上升沿初始化计数器，并且产生一个更新信号

```
TIM_SelectMasterSlaveMode(TIM2,TIM_MasterSlaveMode_Enable); //主从模式选择
```

8.34 简述通用定时器的比较输出过程。

当 CNT 计数值（变动至）与 CCR 值相等时：

- ☐ 相应输出引脚可根据所设置编程模式选择以下赋值（并输出）：置位、复位、翻转或不变；
- ☐ 中断状态寄存器中相应标志位置位；
- ☐ 如果相应中断屏蔽位置位（置 1，即不屏蔽），则产生中断；
- ☐ 如果 DMA 请求使能置位，则产生 DMA 请求。

8.36 参照书中例子，设计一个红绿灯系统，要求：红灯亮 5 秒钟，熄灭，切换绿灯亮 5 秒钟，熄灭，循环往复。给出硬件原理图，并编写代码实现。

可参考书上实例 1-LED 闪烁

引脚可以自己规定，延时可直接用 `delay_ms` 函数也可以用精准延时函数

方法不唯一，可以使用一个引脚，也可以使用两个引脚

注意：需要给出原理图，并且要与代码对应。例如对下面的例子，PA8 置低电平时才会亮。

4. 实例 1-LED 闪烁

1) 功能要求

使目标板上红色 LED 按固定时间一直闪烁。

2) 硬件设计

目标板上红色 LED(LED0)通过一个限流电阻与引脚 PA8 相连，具体电路如图 8.32 所示。



图 8.32 红色 LED(LED0)与 STM32F103 接口电路图

本例，只使用了一个外围设备，发光二极管 LED，嵌入式系统中最为常见的输出设备。

LED 的常见特性：

- ❑ 长脚为阳极，短脚为阴极。
- ❑ 当施以正向/逆向偏压时，LED 发光/不发光。
- ❑ 不同颜色（波长）的 LED，工作电压（即正向电压）不同。如，红色和黄色 LED 的工作电压一般为 1.8~2.2V，蓝色和绿色 LED 一般为 3.0~3.6V。
- ❑ 发光亮度与通过的电流成正比。工作电流一般在几毫安至几十毫安之间，通常为 20mA 左右。若电流过大会损坏 LED，须串联一个限流电阻，如图 8.32 所示。

引脚 PA8 应设置为普通推挽输出 `GPIO_Mode_Out_PP` 工作模式。当 PA8 输出低电平(0V)时，红色 LED(LED0)点亮；当 PA8 输出高电平(3.3V)时，LED0 熄灭。

8.38 在中断优先级中，抢占优先级和子优先级如何发挥各自作用？实际应用中是怎么分组并设置的，以 STM32F103 举例说明。

中断优先级分为抢占优先级和子优先级。

- ❑ 抢占优先级，又称主/组/占先优先级，标识一个中断抢占式优先响应能力高低，决定是否会有中断嵌套发生。如，一个具有高抢占先优先级的中断会打断当前正在执行的中断服务程序（转而执行较高优先级中断所对应 ISR）。
- ❑ 子优先级，又称从优先级，仅在抢占优先级相同时才有影响，标识一个中断非抢占式优先响应能力高低。在抢占优先级相同时：如果有中断正被处理，高子优先级中断须等待正被响应的低子优先级中断处理结束后才能得到响应；如果没有中断正被处理，高子优先级中断将优先被响应。

3) 设置中断优先级

中断优先级的设置通过配置 NVIC 实现，可依次分两步完成：

①设置中断优先级分组的位数。确定在表示中断优先级的 4 位中抢占优先级和子优先级各自所占位数。根据中断总数及是否存在中断嵌套，可有 5 种方式：

- ☐ NVIC_PriorityGroup_0: 抢占优先级 0 位，子优先级 4 位，此时不会发生中断嵌套；
- ☐ NVIC_PriorityGroup_1: 抢占优先级 1 位，子优先级 3 位；即抢占优先级在 0~1（1 位范围 0~1）中取值，子优先级在 0~7（3 位范围 000~111）中取值；以下类推；
- ☐ NVIC_PriorityGroup_2: 抢占优先级 2 位，子优先级 2 位；
- ☐ NVIC_PriorityGroup_3: 抢占优先级 3 位，子优先级 1 位；
- ☐ NVIC_PriorityGroup_4: 抢占优先级 4 位，子优先级 0 位。

②设置中断的抢占优先级和子优先级。根据①中所给出优先级分组情况，分别设置抢占优先级和子优先级，即在各自取值范围中设置优先级的确定值。

8.41 STM32F103 的中断系统共支持多少个异常？其中包括多少个内部异常和多少个可屏蔽的非内核异常中断？

此题说法不够明确，答案仅供参考

STM32F10x 的中断分类如表 8.44 所示，其 NVIC 具有以下主要特性：

- ☐ 支持 84 个中断，包括 16 个内核中断和 68 个可屏蔽非内核中断；
- ☐ 使用 4 位优先级设置，具有 16 级可编程中断优先级；
- ☐ 中断响应/返回时处理器状态的自动保存/恢复均无须额外指令；
- ☐ 支持嵌套和向量中断；
- ☐ 支持中断尾链技术。

8.43 STM32F103 的外部中断/事件线是如何与端口映射的？

1) 外部中断/事件输入线

由图 8.56 可见，EXTI 内部信号线上画有一条斜线，旁边标有 19，表示这样的线路共有 19 根。同样，EXTI 的外部中断/事件输入线也有 19 根，分别为 EXTI0~EXTI18。除 EXTI16（PVD 输出）、EXTI17（RTC 闹钟）和 EXTI18（USB 唤醒）外，其余 16 根外部信号输入线 EXTI0~EXTI15 可分别对应 16 个引脚 Px0~Px15，x 为 A~G。

引脚连接 16 根外部中断/事件输入线的方式如下：任一端口 0 号引脚（如 PA0、PB0、…、PG0）映射到 EXTI 外部中断/事件输入线 EXTI0 上，任一端口 1 号引脚（如 PA1、PB1、…、PG1）映射到 EXTI1 上，…，任一端口 15 号引脚（如 PA15、PB15、…、PG15）映射到 EXTI15 上，如图 8.57 所示。

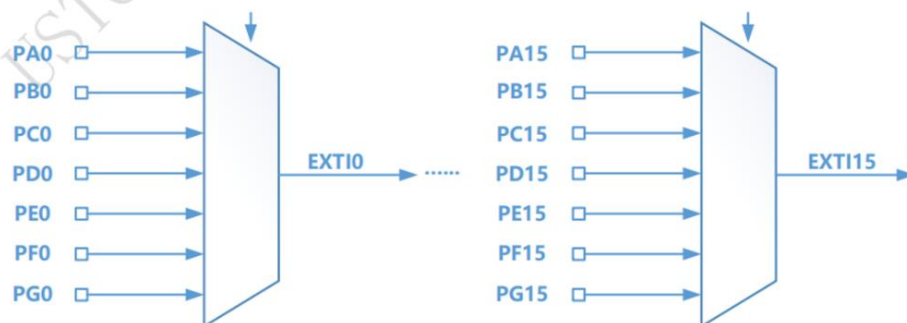


图 8.57 EXTI 外部中断/事件输入线的映像

同一时刻，只能有一个端口（即 A~G 中只选一个）的 n 号引脚映射到 EXTI_n 上，n 为 0~15。如将某 I/O 引脚映射为 EXTI 外部中断/事件输入线，须将该引脚设置为输入模式。

8.45 EXTI 的外部中断/事件请求的产生和传输过程为何？以 STM32F103 为例，具体说明。

1) 外部中断/事件请求的产生和传输

对照图 8.56 中各单元编号①…⑥，从输入（外部输入线）到输出（外部中断/事件请求信号），外部中断/事件请求的产生和传输过程，依次如下：

- ❑ 外部信号从①引脚进入；
- ❑ 经过②边沿检测电路；此电路受上升沿触发选择寄存器和下降沿触发选择寄存器（这两个平行寄存器）控制，可配置选择上升沿、下降沿或双边沿（即同时选择上升沿和下降沿）产生中断/事件；
- ❑ 经过③或门；此或门另一输入是软件中断/事件寄存器，软件可优先于外部信号产生一个中断或事件请求，即当软件中断/事件寄存器对应位为 1 时，不管外部信号如何，或门都会输出有效信号；至此，无论中断或事件，外部请求信号传输路径一致；
- ❑ 外部请求信号进入④与门；此与门另一输入是事件屏蔽寄存器，其对应位为 0，则屏蔽某外部事件，该外部请求信号不能传输到与门另一端；其对应位为 1，则与门产生有效输出并送至⑤脉冲发生器；脉冲发生器把一个跳变信号转变为一个单脉冲，输出到其他功能模块；至此为外部事件请求信号传输路径，如图 8.56 中点划线箭头所示，即①→②→③→④→⑤；
- ❑ 外部请求信号进入请求挂起寄存器（记录外部信号电平变化）；然后进入⑥与门；此与门功能和④与门类似，引入中断屏蔽寄存器控制；仅当其对应位为 1 时，该外部请求信号才被送至中断控制器 NVIC，从而发出一个中断请求，否则，屏蔽之；至此为外部中断请求信号传输路径，如图 8.56 中虚线箭头所示，即①→②→③→⑥。

8.50 简述 STM32F103 的 USART 数据接收/发送过程。

1) USART 数据发送过程

内核指令或 DMA 外设先将数据从内存（变量）写入发送数据寄存器 TDR。然后，发送控制器适时地自动把数据从 TDR 加载到发送移位寄存器，将数据一位一位地通过 Tx 引脚发送出去。当数据完成从 TDR 到发送移位寄存器的转移后，会产生 TDR 已空事件 TXE。其后，当数据从发送移位寄存器全部发送到 Tx 后，会产生数据发送完成事件 TC。可在 SR 中查询这些事件。

- 数据发送，须设置相关寄存器各位，过程如下：
- ①CR1.UE 置位 1，激活 USART；
 - ②CR1.M 定义字长；
 - ③CR2.STOP 定义停止位数；
 - ④若采用多缓冲器通信，配置 CR3.DMAT 使能 DMA，（另外配置 DMA）；
 - ⑤利用 BRR 选择波特率；
 - ⑥置位 CR1.TE，发送一个空闲帧作为第一次数据发送；
 - ⑦将要发送数据写入 DR（此动作会清除 SR.TXE）；一个缓冲区情况下，重复⑦。

2) USART 数据接收过程

是数据发送的逆过程。数据从 Rx 引脚一位一位地输入到接收移位寄存器中。然后，接收控制器自动将接收移位寄存器的数据转移到接收数据寄存器 RDR 中。最后，内核指令或 DMA 将 RDR 数据读入内存（变量）中。当接收移位寄存器的数据转移到 RDR 后，会产生 RDR 非空/已满事件 RXNE。

- 数据接收配置如下（前 5 步与发送相同）：
- ⑥置位 CR1.RE，激活接收器，开始寻找起始位；
 - ⑦接收到一个字符时，SR.RXNE 被置位，表明移位寄存器内容被转移到 RDR，此时若 CR1.RXNEIE=1（即中断使能），则产生中断；接收期间若检测到帧/溢出/噪声错误，相应标志会置位（供查询）。
 - ⑧SR.RXNE 清零：多缓冲器，由 DMA 读 SR 完成；单缓冲模式，软件读 SR 完成；也可通过对其写 0 完成。清零须在下一字符接收结束前完成，避免溢出错误。

8.52 *设计一个简单的双机通信系统，要求：在两个 ARM 处理器之间，用 USART 实现，各自具有收/发缓冲内存区域，协议自定，结束条件自定。给出相关硬件连接图，编写代码实现。

硬件连接图注意 tx 和 rx 交叉连接，也可以用直接使用教材上例子给出的串口芯片收/发缓冲内存区域可以通过定义数组实现
参考书上流程实现单工通信即可

8.61 *解释如下寄存器位的含义。

寄存器位	含义
USART_SR.TXE	发送数据寄存器空 0：数据还未转移到移位寄存器；1：已转移（单缓冲器传输使用）
USART_SR.TC	发送完成 0：发送未完成；1：已完成（当移位寄存器空，且 SR.TXE=1 时）

USART_SR.RXNE	读数据寄存器非空 0: 数据未收到; 1: 收到, 可读
SPI_CR2.TXEIE	发送缓冲空中断使能 0: 禁止; 1: 允许 TXE 中断, TXE 标志置位时产生中断请求
SPI_CR2.RXNEIE	RXNE 中断使能 0: 禁止; 1: 允许中断, RXNE 标志置位时产生中断请求
SPI_SR.TXE	发送缓冲区空 0: 非空; 1: 发送缓冲区空
SPI_SR.RXNE	接收缓冲区非空 0: 空; 1: 接收缓冲区非空

8.62 *解释如下结构体各成员变量的含义。

typedef struct

```
{
    uint32_t USART_BaudRate;           //波特率
    uint16_t USART_WordLength;         //传输字长=数据位数+检验位数, 任选: 8b 或 9b
    uint16_t USART_StopBits;           //停止位数: 1、1_5、2、0_5
    uint16_t USART_Parity;             //校验方式: No 无校验; Odd 奇校验; Even 偶校验
    uint16_t USART_HardwareFlowControl;
    //硬件流控: None 无; RTS 请求发送; CTS 清除发送; RTS_CTS 发送和接收都用流控
    uint16_t USART_Mode;               //串口模式 (可组合): Tx 发送模式; Rx 接收模式
} USART_InitTypeDef;
```

8.63 *在图 4.84 的不同时刻 (如下图所示 T0 时刻、T1 时刻、T2 时刻), I²C 主设备和 I²C 从设备各产生哪些事件? 从 I2C_SR1.SB、I2C_SR1.ADDR、I2C_SR1.STOPF、I2C_SR1.BTF、I2C_SR1.RXNE、I2C_SR1.TXE 中选择。

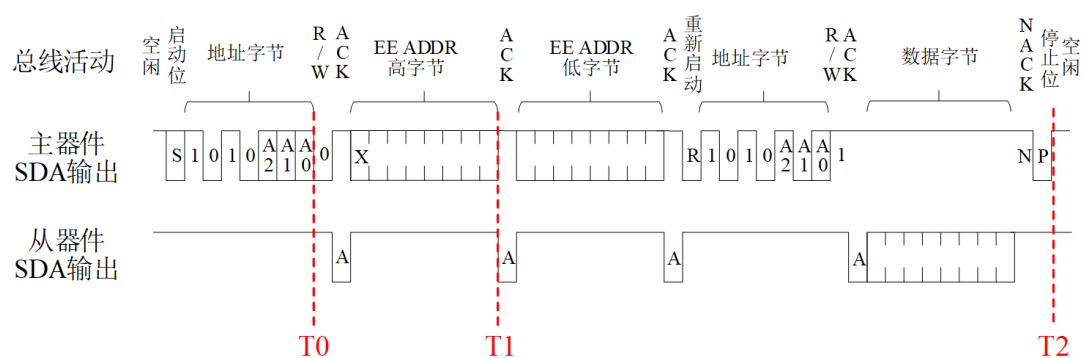


图 4.86 中，dsPIC30F 作主器件，24LC256 作从器件，需要读取的是 24LC256 内指定存储单元中的字节。由于访问存储器需要先发送存储单元地址，然后再传输数据，可以把 dsPIC30F 访问 24LC256 的过程分成四个阶段：①主器件 dsPIC30F 输出 24LC256 芯片的地址（I²C 地址），并发写指令，从器件应答。②主器件 dsPIC30F 输出拟访问的存储单元地址（16 位），从器件 24LC256 应答。③重复启动改变传输方向，主器件 dsPIC30F 输出 24LC256 芯片的地址（I²C 地址），并发读指令，从器件 24LC256 应答。④从器件 24LC256 发出所指定存储单元的内容，主器件应答。

参考资料：STM32——I2C 详解

<https://blog.csdn.net/xiebs/article/details/121467622>

此题答案仅供参考，可能并不完全准确

T0

主：I2C_SR1.ADDR, I2C_SR1.TXE

从：I2C_SR1.ADDR

T1

主：I2C_SR1.TXE

从：I2C_SR1.RXNE

T2

主：I2C_SR1.TXE , I2C_SR1.BTF;

从：I2C_SR1.RXNE , I2C_SR1.STOPF